Model-Agnostic Quality Assessment for LLM-Generated Code via Dynamic Internal Representation Selection

Thanh Trong Vu, Tuan-Dung Bui, Thu-Trang Nguyen, Son Nguyen* and Hieu Dinh Vo

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:
LLM-generated code, code LLMs, code quality assessment, internal representations, white-box, open-box approach

ABSTRACT

Large Language Models (LLMs) have demonstrated impressive capabilities in code generation and are increasingly integrated into the software development process. However, ensuring the quality of LLM-generated code remains a critical concern. Prior work has shown that the internal representations of LLMs encode meaningful signals for assessing code correctness. Nevertheless, the existing methods rely on representations from pre-selected/fixed layers and token positions, which could limit its generalizability across diverse model architectures and tasks. In this work, we introduce AutoProbe, a novel model-agnostic approach that dynamically selects the most informative internal representations for code quality assessment. AutoProbe employs an attention-based mechanism to learn importance scores for hidden states, enabling it to focus on the most relevant features. These weighted representations are then aggregated and passed to a probing classifier to predict code quality across multiple dimensions, including compilability, functionality, and security. To evaluate the performance of AutoProbe, we conduct extensive experiments across multiple benchmarks and code LLMs. Our experimental results show that AutoProbe consistently outperforms the baselines. For security assessment, AutoProbe surpasses the state-of-the-art white-box approach by 19%. For compilability and functionality assessment, AutoProbe demonstrates its highest robustness to code complexity, with the performance higher than the other approaches by up to 19% and 111%, respectively. These findings highlight that dynamically selecting important internal signals enables AutoProbe to serve as a robust and generalizable solution for assessing the quality of code generated by various LLMs.

1. Introduction

In recent years, Large Language Models (LLMs) have exhibited their remarkable capabilities in code generation [1, 2, 3, 4] and are increasingly being integrated into the modern software development process [5, 6, 7]. Especially, Code LLMs, such as DeepSeek Coder [8] or Code Llama [9], are fine-tuned on extensive code corpora, enabling them to effectively leverage both natural language understanding and program synthesis abilities. These models have demonstrated strong performance on various code generation benchmarks [1, 10] and have shown promise in automating a wide range of software engineering (SE) tasks [4].

However, recent empirical studies reveal that code generation performance of LLMs often degrades in practical development scenarios [2, 3, 11, 12]. Although LLMs are capable of generating plausible code, their outputs could be functionally incorrect or insecure [11, 13, 14]. Perry *et al.* [13] report that while AI assistants can significantly improve software development productivity, they can also result in more insecure code. Additionally, Tihanyi *et al.* [15] employed SMT-based Context-Bounded Model Checker to validate LLM-generated code and found that over 62% of the generated programs contain vulnerabilities.

*Corresponding author

thanhvu@vnu.edu.vn (T.T. Vu); 21020006@vnu.edu.vn (T. Bui); trang.nguyen@vnu.edu.vn (T. Nguyen); sonnguyen@vnu.edu.vn (S. Nguyen); hieuvd@vnu.edu.vn (H.D. Vo)

ORCID(s): 0009-0007-7318-6896 (T. Bui); 0000-0002-3596-2352 (T. Nguyen); 0000-0002-8970-9870 (S. Nguyen); 0000-0002-9407-1971 (H.D. Vo)

As LLM-generated code is increasingly integrated into real-world applications, ensuring its correctness and reliability has become critical [16, 17, 18, 19]. Low-quality code can lead to significant functional failures, security flaws, and increased technical debt. These risks demand effective and rigorous evaluation techniques to assess the quality and reliability of code produced by LLMs [20, 21, 22].

To address the challenge of assessing the quality of LLM-generated code, recent studies have explored both *black-box* and *white-box* strategies. Black-box approaches [23, 24, 25] typically rely solely on the final generated outputs. For example, pre-trained models such as CodeBERT [26] and CodeT5 [27] are employed to encode the semantics of the LLM-generated code, and a downstream classifier then predicts whether the code is correct. Meanwhile, white-box approaches [28, 29, 30, 31, 32, 33] leverage the internal reasoning process of the LLM during generations, e.g., hidden states or attention activations, to detect correctness. These methods have demonstrated that internal representations of LLMs encode meaningful signals related to code correctness, enabling the white-box approaches to outperform the black-box ones [28, 29, 32] across multiple benchmarks.

Despite their potential performance, these existing white-box approaches [28, 29, 32] adopt a *rigid representation selection strategy*, which restricts their generalizability and robustness. These approaches assume that correctness-related signals consistently reside in the same layers and token positions. However, in practice, the distribution of these signals is highly dependent on model architectures and tasks. As a result, relying on a *fixed* selection of layer and token positions can lead to the omission of critical information

and result in suboptimal performance. Moreover, since there is no one-size-fits-all strategy across LLMs, a hard-coded selection strategy is less adaptable to the diversity of the models, reducing its effectiveness in real-world scenarios.

This paper introduces AutoProbe, a novel *model-agnostic white-box* framework for assessing the quality of LLM-generated code. In this work, model-agnostic means that AutoProbe can operate on diverse transformer-based LLMs with accessible internal states. Unlike prior studies [28, 29, 32], AutoProbe does not rely on hand-crafted heuristics that prescribe which layers or token positions to use. Instead, AutoProbe learns to automatically identify the most informative hidden representations, regardless of how many layers the model has or how its architecture distributes code quality-related signals.

To effectively capture the most informative signals, AutoProbe employs an attention-based mechanism to learn importance scores over all internal states. This process allows a probing classifier to focus on the most relevant features while down-weighting the irrelevant ones. The weighted representations are then aggregated and passed to a probing classifier to determine whether the generated code is high or low quality. By *dynamically selecting meaningful signals*, AutoProbe improves its robustness and generalizability across diverse LLMs.

To evaluate the performance of AutoProbe, we conduct extensive experiments across multiple benchmarks and a set of seven code LLMs. The experimental results demonstrate that AutoProbe consistently outperforms baseline approaches across all evaluation dimensions. In security assessment, AutoProbe surpasses the state-of-the-art (SOTA) white-box method, Openia, by up to 19% in accuracy. For compilability and functionality assessment, it exhibits exceptional robustness to increased code complexity, achieving performance gains of up to 19% and 111%, respectively. These findings highlight AutoProbe's capability to deliver accurate and reliable code quality assessment across varied tasks, models, and complexity levels.

In brief, this paper makes the following contributions:

- We formalize the task of code quality assessment across multiple dimensions, including *compilability*, *functionality*, and *security*, providing a rigorous foundation for evaluating LLM-generated code.
- We propose AutoProbe, a novel model-agnostic framework that dynamically selects informative internal representations of LLMs during generations for assessing code quality.
- We conduct extensive experiment evaluations demonstrating the robustness and generalizability of AutoProbe across diverse code LLMs and benchmarks.

2. Background and Motivation

2.1. Background

Large Language Models (LLMs), such as GPT-3 [34], PaLM [35], and LlaMA [36], are typically transformer-based models with hundreds of millions to billions of parameters [37]. These models are pre-trained on massive text corpora using an autoregressive objective, i.e., predicting the next token given the preceding context, to model the probability distribution over token sequences. This training paradigm enables LLMs to understand and generate coherent text, allowing them to perform a wide range of natural language tasks, like translation [38], summarization [39], and question answering [40]. To extend their capabilities beyond natural language, LLMs have been trained or fine-tuned on large-scale code repositories, giving rise to a specialized class known as **code LLMs**, such as DeepSeek Coder [8], Magicoder [41], Code Llama [9]. These models are capable of automating various SE tasks [4, 42, 2, 43], including code generation and code completion.

Similar to general-purpose LLMs, code LLMs generate code output through an autoregressive decoding procedure. Given an input sequence $x = \{x_1, \dots, x_n\}$, which may consist of natural language instructions, code comments, and/or partially written code, the code LLM $\mathcal{M}$ produces a code output sequence $c = \{c_1, \dots, c_m\}$ one token at a time. At each generation step s, the model computes the probability distribution over possible next token c_s, conditioned on the input x and previously generated tokens $c_{<s>} = \{c_1, \dots, c_{s-1}\}$, as follows:

$$P(c_s \mid x, c_{<s>}) = \text{softmax}(W_o \cdot h_{l,s} + b_o) \tag{1}$$

In Eq. 1, $h_{l,s} \in \mathbb{R}^d$ denotes the *internal representation* (or hidden state) of the model at layer l for the token position s, and W_o, b_o are learned output projection weights.

Definition 1 (Internal representation). An internal representation $h_{l,s} \in \mathbb{R}^d$ is a d-dimensional vector that encodes the model's latent representation for token c_s at layer l. It is recursively computed from the hidden state at the previous layer $h_{l-1,s}$ and contextual information $context_s$ via a layer-specific transformation function $f_l(\cdot)$:

$$h_{l,s} = f_l(h_{l-1,s}, context_s) \tag{2}$$

The transformation function $f_l(\cdot)$ in Eq. 2 typically consists of a combination of multi-head self-attention, feedforward networks, and normalization operations. The internal representations form the core computational states of the model, capturing essential information at different levels of abstraction, including syntactic structure, semantic meaning, and task-specific context, that are required for token prediction at each generation step.

With their impressive capabilities, LLM-generated code is increasingly being integrated into real-world applications. However, the quality of such code remains a major concern. *Low-quality code,* such as non-compilable, functionally incorrect, or insecure code, can potentially cause the program

to crash, fail to meet requirements, or become vulnerable to attacks [28, 44]. Prior studies [28, 45, 33, 46] have shown that the model's internal representations encode signals relevant for assessing code quality. Since these internal states reflect the model's evolving understanding and confidence during generation, they can serve as valuable indicators of uncertainty or inconsistency. By analyzing the internal representations, it is possible to estimate the quality of the generated code with a certain degree of accuracy.

2.2. Code Quality Assessment Problem Formulation

Given the prevalence of hallucinations in LLM-generated code and the potential of internal representations to signal such failures, we aim to formulate the task of *code quality assessment* based on a model's internal states.

Let $\mathscr{M}$ be a code LLM consisting of L layer, which generates sequence of code tokens $c = \{c_1, \dots, c_m\}$ given an input $x = \{x_1, \dots, x_n\}$. At each generation step s, $s \in [1, m]$, the model produces internal representation $h_{l,s}$ at every layer l, $l \in [1, L]$, capturing contextualized information about token c_s. We define $\mathcal{H} = \{h_{l,s} \mid l \in [1, L],\ s \in [1, m]\}$ as the set of internal representations associated with the entire generated sequence, across all L layers and m token positions.

Formally, we define the *code quality assessment* task as a binary classification problem:

Definition 2 (Code Quality Assessment). Given the internal representations $\mathcal{H}$ associated with the generated code sequence c, the goal of code quality assessment is to determine whether c satisfies the required quality criteria. Formally, we aim to learn a function:

$$f_\theta : \mathcal{H} \mapsto y$$

where f_θ maps the internal states $\mathcal{H}$ to a quality label $y \in \{0, 1\}$, where $y = 1$ indicating high-quality code and $y = 0$ indicating low-quality (e.g., uncompilable, incorrect, or insecure) code.

The quality of the generated code can be evaluated along multiple dimensions, including *compilability*, *functionality*, and *security*. Depending on the evaluation scenario, different oracles, such as compilers, dynamic test cases, or static security analyzers, can be employed to provide ground-truth quality labels.

Definition 3 (Compilability). Compilability refers to whether the generated code c can be parsed and compiled successfully without syntax or structural errors. Let $\texttt{compile}(c)$ denote the result of compiling c using a standard compiler (e.g., `gcc` for C/C++, `javac` for Java) , where $\texttt{compile}(c) \in \{success, failure\}$. Code c is considered compilable if and only if it compiles without errors:

$$\chi(c) = \begin{cases} 1 & \text{if } \texttt{compile}(c) = success, \\ 0 & \text{otherwise.} \end{cases}$$

Table 1
The numbers of functional correct (*#Cor.*) and incorrect (*#Inc.*) solutions of benchmarks

Code LLM		*HumanEval*	*MBPP*
DeepSeek Coder-*6.7B*	*#Cor.*	1,294	3,033
	#Inc.	346	1,749
Code Llama-*7B*	*#Cor.*	644	1,942
	#Inc.	996	2,686
Magicoder-*7B*	*#Cor.*	1,198	3,135
	#Inc.	440	1,752

Definition 4 (Functionality). Functionality ensures that the generated code c satisfies the functional requirements specified by the input prompt x. Let $\mathcal{T} = \{t_1, \dots, t_k\}$ denote a set of functional test cases designed to validate whether c fulfills the intended behaviors. Let $t(c)$ represent the outcome of applying a test $t \in \mathcal{T}$ to the code c, where $t(c) \in \{passed, failed\}$. Code c is considered functionally correct if and only if it passes all tests in $\mathcal{T}$:

$$\eta(c) = \begin{cases} 1 & \text{if } \forall t \in \mathcal{T},\ t(c) = passed, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

Definition 5 (Security). Security assesses whether the generated code c adheres to secure coding practices. Let $\mathcal{V} = \{v_1, \dots, v_h\}$ denote a set of security vulnerability checks, and $v(c)$ represents the outcome of evaluating c against vulnerability check $v \in \mathcal{V}$, where $v(c) \in \{safe, unsafe\}$. Code c is considered secure if and only if it passes all security checks in $\mathcal{V}$:

$$\sigma(c) = \begin{cases} 1 & \text{if } \forall v \in \mathcal{V},\ v(c) = safe, \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

This work focuses on three key dimensions of code quality: compilability, functionality, and security, as they represent fundamental aspects of practical code quality [47]. These dimensions cover syntactic validity, behavioral correctness, and safety, each critical for ensuring reliable and trustworthy code. While our framework is tailored to these dimensions, it can be readily extended to incorporate additional code quality criteria in a similar manner.

2.3. Motivation

Inspired by the procedure in Openia [28], we conducted an empirical study to assess the functional correctness of code generated by DeepSeek Coder-6.7B, Code Llama-7B, and Magicoder-7B using their internal representations. For HumanEval [1] and MBPP [10] benchmarks, we employed each studied model to generate 10 solutions per task. The functional correctness of each solution is determined by using the provided test cases; these resulting datasets are summarized in Table 1.

For each code LLM, we trained a probing classifier on its internal representations extracted from different layers and

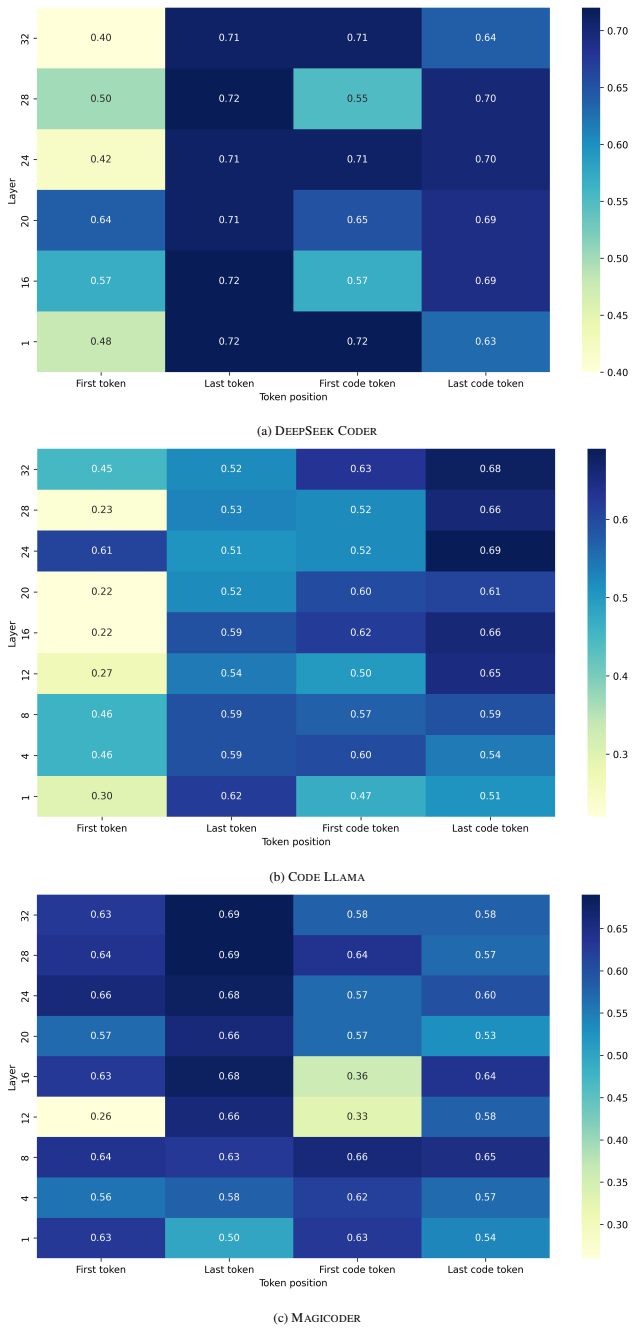


Figure 1: The performance in F1-score of functionality assessment using internal states from different tokens and layers across various code LLMs

token positions during code generation for MBPP tasks. The classifier was subsequently evaluated on its ability to predict the correctness of the solutions for HumanEval tasks. In this experiment, representations were extracted from four specific token positions: `first token` and `last token` (marking the start and end of the full generated sequence), and `first code token` and `last code token` (marking the start and end of the code snippet itself).

Figure 1 shows the performance of probing classifiers using internal representations from various layers and token positions for the studied model. The results reveal a key challenge: ***the code correctness assessment performance varies significantly across models, layers, and token positions***.

First, the choice of token position plays a crucial role in the predictive performance of internal representations, with notable variation across models. For instance, in DeepSeek Coder and Magicoder, the highest performance is achieved using the internal representations of the `last token`, while in Code Llama, the best results are observed from the hidden states of the `last code token`. Conversely, the representations at `first token`, while being the least informative for both DeepSeek Coder and Code Llama, provide comparatively strong signals in Magicoder.

Moreover, the predictive performance varies significantly across layers, even for the same token position. In Magicoder, the performance improves with layer depth, i.e., the F1-score for the `last token` at the `final layer` reaches 0.69, which is 40% higher than that at the `first layer`. In contrast, DeepSeek Coder exhibits remarkably stable performance across layers, maintaining a high F1-score for the `last token` representations. Code Llama shows highly inconsistent patterns. For the `last token`, better performance is observed at `early and middle layers`, while for the `last code token`, the best F1-scores are achieved using representations from the `middle and final layers`.

These findings suggest that the optimal internal representations for code correctness assessment are highly model-dependent, with the most effective layers and token positions varying significantly across models. In other words, *no single layer or token position consistently captures correctness signals across all models*. This highlights the need for a general solution that can ***dynamically and automatically select the most informative internal states***, enabling reliable correctness assessment in a model-agnostic manner.

3. Model-Agnostic Quality Assessment for LLM-Generated Code via Dynamic Internal Representation Selection

3.1. Approach Overview

This paper proposes AutoProbe, a novel model-agnostic approach for assessing the quality of LLM-generated code by dynamically identifying informative signals from LLMs' internal states during code generation. Rather than relying on a fixed or manually engineered "one-size-fits-all" heuristic that prescribes which layers or token positions should be used, AutoProbe learns to adaptively determine the relevance of internal representations for code quality prediction. By leveraging a data-driven attention-based mechanism, AutoProbe adjusts to the internal signal distributions exhibited by different LLMs, enabling robust and generalizable code quality assessment across models. Importantly, AutoProbe does not rely on any prior assumption or heuristic regarding where code quality-related signals should reside (e.g., specific layers, token positions, or architectural blocks). Instead, it infers useful states directly from data,

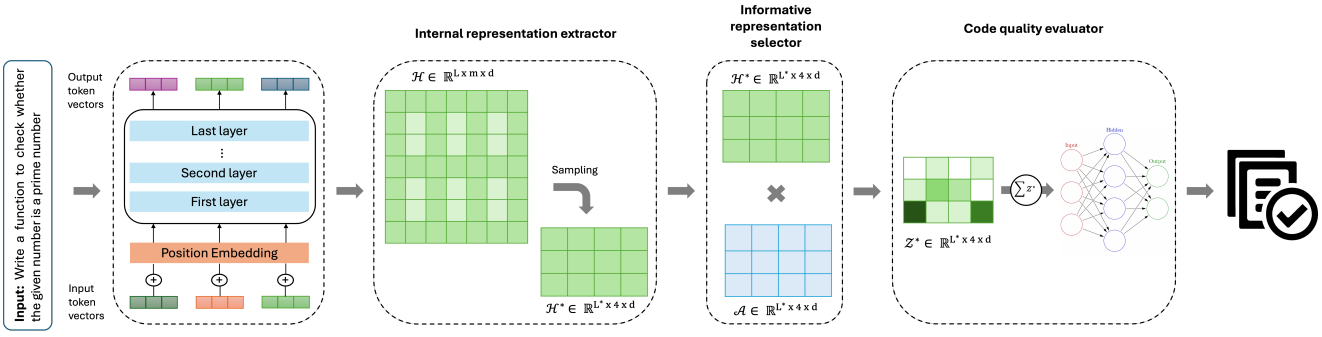


Figure 2: Approach overview

allowing AUTOPROBE to remain model-agnostic, i.e., capable of operating on diverse transformer-based LLMs with accessible internal states, without requiring any modification to AUTOPROBE itself.

As shown in Figure 2, AUTOPROBE comprises three key components: *Internal Representation Extractor*, which gathers hidden states from multiple layers and token positions; *Informative Representation Selector*, which prioritizes the most relevant representations based on the attention mechanism; and *Code Quality Evaluator*, which employs a probing classifier to determine whether the generated code meets the required quality criteria based on the dynamically selected features.

3.2. Internal Representation Extractor

This module collects internal representations produced during code generation as inputs for subsequent components of AUTOPROBE. However, the comprehensive set of $L \times m$ internal representations for the entire code sequence c length m, $\mathcal{H} = \{h_{l,s} \mid l \in [1, L],\ s \in [1, m]\}$ is computationally expensive to process and potentially noisy for code quality assessment. Moreover, as illustrated in our empirical study, not all tokens and layers contribute equally to the code quality assessment, depending on both the model and the generation context. A naive approach using all representations would therefore be inefficient and could degrade performance.

To improve computational efficiency without sacrificing predictive performance, *Internal Representation Extractor* adopts a sampling-based strategy that extracts a compact yet expressive subset of states $\mathcal{H}^* \subset \mathcal{H}$. This is achieved via a two-pronged strategy that selects representations based on their token and layer positions.

Token Position Sampling. For token sampling, rather than uniformly sampling tokens or relying on fixed intervals, we adopt a *boundary-aware token selection heuristic* guided by both the structural and semantic considerations. Specifically, we select four *structurally and semantically significant* positions: the `first` and `last tokens` of the entire generated sequence, and the `first` and `last code tokens` that mark the actual code snippet. The `first` and `last` tokens often reflect the model's overall generation context, as well as its uncertainty or confidence [48, 49]. Meanwhile, the `first` and `last` code tokens typically carry essential information related to syntax, control flow, and output structure [28].

In practice, token positions can also be selected using other strategies such as syntax-aware analysis, rule-based heuristics, or model-specific indicators. However, we employ this boundary-aware heuristic for several reasons. First, prior studies have shown that boundary tokens often carry stronger quality-related signals than interior positions [49]. Second, this strategy yields a fixed number of token positions regardless of the sequence length, which varies significantly across tasks. This consistency simplifies downstream processing and model training. Third, the heuristic is lightweight and adaptable to different code formats.

Layer Sampling. To avoid redundantly extracting hidden states from all L layers, we apply a uniform sampling strategy over layers. Specifically, we sample layers at a fixed interval k, starting from the first layer. That is, we select layers at positions $l \in \{1, 1+k, \ldots, 1+\lfloor\frac{L-1}{k}\rfloor.k\}$. This results in a total of $L^* = \lfloor\frac{L}{k}\rfloor$ sampled layers. This sampling balances the trade-off between representation diversity and computational complexity.

By restricting the extraction to a compact set of L^* layers and four token positions, we obtain a compact yet expressive set of internal states $\mathcal{H}^* \subset \mathcal{H}$. $\mathcal{H}^*$ serves as input to the subsequent selector module, which identifies the most informative states for final prediction. Although token and layer sampling may introduce the risk of omitting potentially useful internal states, our empirical results in Sec. 5.2.1 demonstrate that this strategy achieves strong predictive performance while substantially reducing computational overhead. These findings validate the effectiveness and efficiency of our sampling approach in practice.

3.3. Informative Representation Selector

To identify the most informative internal states within the compact set $\mathcal{H}^*$, AUTOPROBE employs the *attention-based mechanism* that learns to dynamically prioritize representations based on their relevance to code quality assessment. Particularly, in *Informative Representation Selector*, the attention mechanism assigns a learned importance score to each internal state, quantifying its contribution to the final prediction. By aggregating the internal states weighted by these scores, the code quality evaluator can

emphasize the most informative signals while effectively down-weighting irrelevant ones, enabling general and robust assessment across diverse code LLMs.

Specifically, the input of the informative representation selector module is $\mathcal{H}^* = \{h_{l,i} \mid l \in \{1, 1+k, \ldots, 1+\lfloor\frac{L-1}{k}\rfloor.k\}, i \in S\}$, where S denotes the set of four selected token positions, $S = \{$`first token`, `last token`, `first code token`, `last code token`$\}$. The selector module assigns a weighted score (attention score) $\alpha_{l,i}$ to each internal state $h_{l,i}$, indicating its importance for quality assessment. The weighted representation is then computed as:

$$z_{l,i} = \alpha_{l,i} \cdot h_{l,i} \tag{5}$$

The collection of all the weighted vectors $z_{l,i}$ forms the output set $\mathcal{Z}^*$, which summarizes the weighted contributions of the internal states in $\mathcal{H}^*$ to the final code quality prediction.

To compute attention scores $\alpha_{l,i}$, each internal state in $\mathcal{H}^*$ is linearly projected and normalized via softmax:

$$\alpha_{l,i} = \text{softmax}(H \cdot W_a + b_a)_{l,i} \tag{6}$$

In Eq. 6, $H \in \mathbb{R}^{(L^* \times 4) \times d}$ is the flattened matrix formed by stacking all vectors in $\mathcal{H}^*$. The parameters $W_a \in \mathbb{R}^{d \times 1}$ and $b_a \in \mathbb{R}$ are learnable weights that define the attention scoring function. The softmax ensures that attention scores across all positions sum to 1, enabling a normalized weighting over the entire set $\mathcal{H}^*$, i.e., $\sum_{l,i} \alpha_{l,i} = 1$. All the attention scores form an attention sets $\mathcal{A} = \{a_{l,i}\}$.

In AutoProbe, the parameters W_a and b_a are jointly trained with the code quality evaluator using supervised learning. During training, the model receives examples of LLM-generated code along with their quality labels, which are assigned according the evaluation criteria, e.g., compilability or functionality (Sec. 2.2). The cross-entropy loss is measured and back-propagated through the attention mechanism, enabling the model to learn which internal states are most indicative of code quality.

3.4. Code Quality Evaluator

In *Code Quality Evaluator*, AutoProbe transforms the set weighted representations $\mathcal{Z}^* = \{z_{l,i}\}$ into a single, fixed-length vector of representation. Specifically, it aggregates all the weighted representations in $\mathcal{Z}^*$ into a unified vector $z \in \mathbb{R}^d$ as follows:

$$z = \text{aggregate}(\mathcal{Z}^*) \tag{7}$$

Without loss of generality, the aggregation function *aggregate* in Eq. 7 could be implemented using various strategies such as concatenation, summation, mean/max pooling, etc. The impact of different aggregation methods is empirically evaluated in Sec. 5.2.3.

The aggregated vector z is then fed into a probing classifier that predicts the quality label of the corresponding code c. The probing classifier can be implemented as either a simple neural network or a linear model. It is trained end-to-end with the selector using standard supervised learning. The influence of different classifier architectures on prediction performance is also evaluated in Sec. 5.2.3.

4. Evaluation Methodology

To evaluate the performance of AutoProbe, we seek to answer the following research questions:

RQ1. Performance Analysis: How effective is AutoProbe in assessing the quality of code generated by different code LLMs compared to SOTA methods?

RQ2. Intrinsic Analysis: How do different components of AutoProbe contribute to its overall performance?

RQ3. Sensitivity Analysis: How sensitive is AutoProbe's performance to different external factors such as programming languages, code LLMs, and training data sizes?

RQ4. Efficiency Analysis: How is AutoProbe's time complexity and memory usage?

4.1. Dataset Construction

To evaluate the performance of AutoProbe and the baselines, we construct experimental datasets from several popular code-generation benchmarks, including *HumanEval* [1], *MBPP* [10], *DevEval* [12], *SecurityEval* [50], *CWEval* [51], *CODEGUARD+* [52], and *SALLM* [53]. For each task in these benchmarks, we employed each studied code LLMs to generate 10 candidate solutions.

In particular, *HumanEval*, *MBPP*, and *DevEval* focus on evaluating functional correctness and are therefore used to construct datasets for evaluating *compilability* and *functionality*. *HumanEval* consists of 164 hand-crafted programming tasks, each paired with unit tests to measure functional correctness of LLM-generated code. *MBPP* contains 500 crowd-sourced Python programming tasks with associated test cases, covering a wider range of programming scenarios. *DevEval* includes 1,825 repository-level tasks drawn from real-world software projects, where correctness depends on multi-file dependencies and project-level context.

The remaining benchmarks, *SecurityEval*, *CWEval*, *CODEGUARD+*, and *SALLM* are specifically designed to evaluate security aspects of code generated by LLMs. *SecurityEval* includes 121 tasks derived from security-relevant patterns. *CWEval* provides 25 tasks aligned with common weakness enumerations (CWEs). *CODEGUARD+* contains 71 tasks curated from real-world vulnerable code patterns. *SALLM* offers 100 adversarial security tasks targeting unsafe or exploitable model outputs.

Following the procedures in the prior work [28], the benchmark guidelines, and our criteria defined in Sec.2.2, we assign labels to each generated solution as follows:

- *Compilability*: A solution is labeled *compilable* if it is successfully executed by the Python 3 interpreter; otherwise, it is labeled *uncompilable*.
- *Functionality*: A solution is labeled *correct* if it passes all provided test cases for the corresponding task; otherwise, *incorrect*.
- *Security*: A solution is labeled *secure* if it passes all checks from two widely used static analyzers, Bandit [54] and CodeQL [55]; otherwise, *insecure*.

Table 2
The number of compilable/uncompilable solutions in *Compilability* and the nummber of correct/incorrect solutions in *Functionality* across benchmarks

Criteria	Code LLM		*HumanEval*	*MBPP*	*DevEval*
Compilability	DeepSeek Coder-1.3B	#Compilable	1,464	4,272	5,952
		#Uncompilable	176	494	11,377
	DeepSeek Coder-6.7B	#Compilable	1,577	4,635	8,191
		#Uncompilable	63	147	9,139
	Code Llama-7B	#Compilable	1,479	3,804	7,254
		#Uncompilable	161	824	10,076
	Code Llama-13B	#Compilable	1,385	4,128	5,440
		#Uncompilable	255	792	8,480
	Code Gemma-7B	#Compilable	1,315	4,086	5,049
		#Uncompilable	325	864	8,870
	Magicoder-7B	#Compilable	1,572	4,745	7,432
		#Uncompilable	66	142	9,898
Functionality	DeepSeek Coder-1.3B	#Correct	872	2,065	2,092
		#Incorrect	768	2,701	15,237
	DeepSeek Coder-6.7B	#Correct	1,294	3,033	4,321
		#Incorrect	346	1,749	13,009
	Code Llama-7B	#Correct	644	1,942	3,467
		#Incorrect	996	2,686	13,863
	Code Llama-13B	#Correct	666	2,065	2,849
		#Incorrect	974	2,855	14,481
	Code Gemma-7B	#Correct	718	2,141	1,876
		#Incorrect	922	2,809	12,043
	Magicoder-7B	#Correct	1,198	3,135	4,113
		#Incorrect	440	1,752	13,217

Table 3
The number of secure/insecure solutions in *Security* across benchmarks

Code LLM		*SecurityEval*	*CWEval*	*CODEGUARD+*	*SALLM*
DeepSeek Coder-1.3B	#Secure	482	168	200	231
	#Insecure	728	82	510	769
DeepSeek Coder-6.7B	#Secure	508	168	267	342
	#Insecure	702	82	443	658
Code Llama-7B	#Secure	882	221	506	750
	#Insecure	328	29	204	250
Code Llama-13B	#Secure	675	225	519	545
	#Insecure	535	25	191	455
Code Gemma-7B	#Secure	597	175	289	421
	#Insecure	613	75	421	579
Magicoder	#Secure	598	192	262	451
	#Insecure	612	58	448	549

Table 2 and Table 3 summarize the numbers of correct and incorrect code generated by each studied model for the given benchmarks and criteria. The complete dataset, including generated code, the corresponding internal representations extracted from the studied code LLMs, and their quality labels, could be found on our website [56].

4.2. Studied Code LLMs

To enable a fair, reproducible, and practical evaluation, we selected the studied LLMs based on several key criteria. First, *access to the model's hidden states is essential*, as our goal is to evaluate code quality by leveraging these internal representations. This requirement is typically met by open-source models, whereas closed-models or commercial models often restrict such access. Second, *we focus on code-specialized LLMs*, which are trained or fine-tuned on large-scale code corpora. These models generally outperform general-purpose LLMs on programming tasks, leading to a more realistic and meaningful evaluation. Third, *we consider the diversity of model architectures, model sizes, and training strategies* to broaden the scope of our analysis and improve the generalizability of our findings. Finally, *we limit the model size to a maximum of 20 billion parameters* to accommodate hardware resource constraints and ensure that our evaluation is practical and reproducible.

Based on these criteria, we selected seven representative LLMs from different model families to ensure both architectural diversity and broad generalizability of our evaluation, including DeepSeek Coder-1.3B [8], DeepSeek Coder-6.7B [8], Code Llama-7B [9], Code Llama-13B [9], Magicoder-7B [41], Code Gemma-7B [57], and GPT-OSS-20B [58]. All selected models are open-source, allowing full access to their internal representations during code generation. These models are widely adopted in both academic research and industrial applications, and have demonstrated strong performance on SE tasks [4, 11, 28]. Each employed model is either trained or fine-tuned on large-scale code corpora to capture the syntactic, semantic, and logical aspects of programming languages. For consistency, we use the instruction-tuned versions with officially released pre-trained weights from HuggingFace or their websites and do not perform further fine-tuning. This setup enables a fair assessment of each model's generalization ability and facilitates a controlled comparison of how their internal representations relate to code quality.

4.3. Baselines

We compare the performance of AutoProbe against SOTA *white-box* and *black-box* approaches. White-box approaches analyze the model's internal computations during code generation to determine the quality of the generated output. In contrast, black-box approaches evaluate the code quality based solely on the final generated output without considering the generation process.

- *White-box methods* [28, 33, 59]: These methods utilize internal signals such as attention maps, hidden activations, or internal representations as indicators of code quality. In this study, we evaluate the effectiveness of AutoProbe by comparing it against a SOTA white-box method Openia [28], and an Oracle baseline.
  - Openia [28] determines if LLM-generated code is (in)correct by training a probing classifier using the internal states from the last generated token at the last layer of the models.
  - Oracle represents an upper-bound baseline constructed by exhaustively searching across all layers and token positions to identify the *most optimal* internal representations of each model for each evaluation setting. Since the search space grows exponentially with the number of layers and tokens, such a full search is computationally prohibitive and infeasible in practice. In this research, we approximate this oracle by individually evaluating representations from four boundary tokens (`first token`, `last token`, `first code token`, `last code token`) across

layers, providing a manageable yet informative estimation of the optimal choice.

- *Black-box methods* [28, 23, 24, 25, 29]: We consider two representative black-box strategies:
  - *Post-hoc classification-based methods* [23, 24, 25]: These methods assess code quality by analyzing the code semantics, typically captured using a pretrained model for code, e.g., CodeBERT [26], CodeT5 [27], or CuBERT [60]. In this work, we employed two widely used models, including CodeBERT and CodeT5+, to encode the code semantics and train a classifier on these embeddings to distinguish between high and low quality outputs.
  - *LLM-as-A-Judge (LLM-AJ)* [28, 29]: These methods evaluate code quality by utilizing the reasoning and generalization capabilities of LLMs. Following the prior work [28, 29], we adopt a zero-shot prompting strategy with two configurations: In-house LLM-AJ, where the same model, that generates code, is used to assess its own output, and External LLM-AJ, where an external LLM (i.e., o4-mini) serves as a verifier. The full prompt templates used in our experiments are available on our website [56].

4.4. Evaluation Procedure

RQ1. Performance analysis: We evaluate AutoProbe and the baselines across three key dimensions: *compilability*, *functionality*, and *security* under different settings:

- ***Independent-unit code generation*:** This setting focuses on standalone programming tasks [1, 10, 50, 51, 52, 53], where each prompt provides all necessary information for solving the problem. The benchmarks used in this category, including *HumanEval* [1], *MBPP* [10], *SecurityEval* [50], *CWEval* [51], *CODEGUARD+* [52], and *SALLM* [53], contain short, self-contained code snippets that do not rely on external files or project-level context.

- ***Repo(sitory)-level code generation*:** This setting evaluates code quality within the context of real-world software projects. The *DevEval* benchmark [12] represents this category, where tasks require LLMs to reason over multi-file structures and preserve cross-file consistency. Such tasks introduce substantially higher complexity, as the model must generate or modify code while maintaining dependencies and contextual relationships across the repository [2, 3]. This setting therefore measures the robustness of different approaches in handling context-dependent code generation scenarios.

AutoProbe, OpenIA, and the post-hoc classification-based methods all require training a classifier. To ensure a fair comparison, we adopt the same training sets for training classifiers of these approaches and evaluate all the methods on the same test sets. We first group benchmarks by evaluation setting. For *independent-unit* functionality and compilability, we combine *MBPP* and *HumanEval*. For *independent-unit* security, we merge *SecurityEval*, *CWEval*, *CODEGUARD+*, and *SALLM*. For *repo-level* functionality and compilability, we use *DevEval*. Each combined dataset is then randomly split into training and testing subsets using a 9:1 ratio. For the post-hoc classification-based methods, the classifier is trained on the *code snippets generated* for tasks in the training set. Meanwhile, OpenIA and AutoProbe train their classifiers using the *internal representations collected during code generation* for the same training tasks.

Furthermore, we use the same architecture for the classification models in the post-hoc methods, OpenIA, and AutoProbe. Each classifier contains an input layer, two hidden layers with 128 and 64 neurons respectively, and an output layer. All models are trained for 50 epochs using a batch size of 32 and a learning rate of 10^{-3}.

RQ2. Intrinsic Analysis: We investigate how different components of AutoProbe contribute to its overall performance. To answer this question, we systematically evaluate AutoProbe under different configurations.

For the *Internal Representation Extractor*, we aim to examine the impact of different sampling strategies:

- *Token sampling*: We compare AutoProbe's performance under three sampling strategies: (1) *full token*, (2) *random*, and (3) *boundary-aware*. *Full token* uses representations from all generated tokens. To accommodate varying sequence lengths, we fix the sequence size to 256 tokens and apply padding or truncation as needed. *Random* randomly selects one interior token position. *Boundary-aware* selects four key tokens at boundary positions (Sec. 3.2).

- *Layer sampling*: We evaluate the impacts of sampling different subsets of layers by varying the sampling intervals, $k \in \{1, 2, 3, 4, 5\}$, which defines the step size between selected layers.

For the *Informative Representation Selector*, we conduct an ablation study to investigate its contribution to AutoProbe's performance. Specifically, we compare AutoProbe's performance under two configurations:

- *With Selector*: This module is *enabled* to assign importance scores to each internal representation.

- *Without Selector*: The selector module is *disabled*. All internal representations are uniformly aggregated with equal importance.

For the *Code Quality Evaluator*, we study how the choice of aggregation function and probing classifier affects the overall results. Regarding the *aggregation function*, we evaluate several strategies for combining the selected internal representations $\mathcal{Z}^*$, including concatenation, summation, mean pooling, and max pooling. For the *probing classifier*,

we explore various classifier architectures, including Multi-Layer Perceptron (MLP), Support Vector Machine (SVM), and Logistic Regression.

All intrinsic analysis experiments are conducted using Code Gemma-7B on the MBPP benchmark. For each experiment, the MBPP tasks are randomly split into a 90:10 ratio for training and testing. Internal representations extracted during the code generation process for the tasks in the training set are used to train the probing classifier, while those from the test set are used to evaluate its performance.

RQ3. Sensitivity analysis: This experiment examines AutoProbe's generalization capability across programming languages and LLMs. Specifically, we assess whether AutoProbe, when trained on internal representations extracted from code generation in certain programming languages, can accurately predict the quality of code written in a completely different language. We also analyze how varying the size of the training data affects AutoProbe's prediction performance. Additionally, we examine AutoProbe's ability to transfer across models by training it on the internal states of several code LLMs and testing its performance on code generated by a different model.

4.5. Metrics

Determining the quality of code generated by an LLM can be formulated as a binary classification problem, i.e., classifying whether a code snippet is high or low quality according to criteria such as compilability, functionality, and security. To evaluate AutoProbe and the baseline methods, we employ standard classification metrics. For a fair evaluation across both classes, especially in the presence of class imbalance, we adopt weighted metrics including *Accuracy*, *Weighted Precision*, *Weighted Recall*, and *Weighted F1-Score*. These metrics compute a label-wise average weighted by the proportion of each class in the test set. We also evaluate the efficiency of each approach by measuring time complexity (for both training and inference) and memory usage. These metrics provide a comprehensive understanding of the practical feasibility of the approaches.

5. Experimental Results

5.1. Performance Analysis

5.1.1. Security Assessment

The performance of AutoProbe and the baseline approaches in code quality assessment under the security criterion is shown in Table 4. Overall, AutoProbe *consistently outperforms all baselines across all studied code LLMs. Notably,* AutoProbe*'s performance closely matches or even exceeds that of* Oracle*, indicating its effectiveness in capturing informative internal signals.*

Compared to the white-box approach Openia, AutoProbe yields improvement gains of up to 19%. For example, when assessing code generated by GPT-OSS-20B, AutoProbe reaches an accuracy of 0.89, while this figure of Openia is only 0.75. This performance gap can be attributed to the limitations of Openia's design, which relies on fixed representations from the last generated token at the last layer of the LLM. Such a rigid strategy hinders its flexibility and generalizability across different model architectures. In contrast, AutoProbe dynamically selects the most informative internal states, allowing it to tailor the selected representations to each model's characteristics. This flexible mechanism enables AutoProbe to capture security-related signals more effectively, resulting in more robust performance across diverse models.

Both In-house LLM-AJ and External LLM-AJ achieve considerably low performance. The accuracy of In-house LLM-AJ lags behind AutoProbe by margins ranging from 24% to 179%. In the In-house LLM-AJ setting, the same LLM that generates the code is also used to assess its own outputs, making it prone to overconfidence in the quality of its responses. Similarly, External LLM-AJ falls short of AutoProbe by 19% to 144%, even when using a powerful external model like o4-mini. This performance gap underscores the challenge of detecting security vulnerabilities, which often require deep semantic understanding and context-aware reasoning. These results expose the limitations of using LLMs directly as code quality judges under the security criterion.

The black-box classification approaches based on CodeBERT and CodeT5+ show quite strong performance across all code LLMs in evaluating the security criterion of code. For instance, the accuracies achieved by CodeT5+ are comparable to Openia and better than External LLM-AJ by up to 55%. However, their performance still lags behind AutoProbe from 5% to 14%.

Furthermore, AutoProbe exhibits strong performance, which is close to Oracle. Interestingly, on DeepSeek Coder-1.3B, AutoProbe achieves an accuracy of 0.90, which slightly surpasses Oracle's performance. By aggregating the sampled and weighted internal representations, AutoProbe is able to not only focus on the most informative states but also integrate complementary information across multiple tokens and layers. This mechanism enhances its ability to detect security vulnerabilities. As a result, AutoProbe can outperform Oracle, which relies on a single representation in some cases. These results highlight the effectiveness of AutoProbe in capturing important signals for security prediction.

5.1.2. Functionality Assessment

Table 5 shows that External LLM-AJ *is the most effective approach for assessing the functionality correctness of independent-unit code, while* AutoProbe *performs best in the repo-level code setting.* Specifically, in the *independent-unit* setting, External LLM-AJ achieves an average accuracy of 0.94, exceeding that of the other approaches by 32–121%. In the *repo-level* setting, AutoProbe reaches an average accuracy of 0.81, which is comparable to the Oracle's performance and surpasses the other approaches by up to 111%.

The performance of LLM-AJ approaches, including both In-house LLM-AJ and External LLM-AJ, *declines*

Table 4
Code quality assessment performance in ***security*** criterion

Code LLM		Accuracy	Precision	Recall	F1-Score
DeepSeek Coder-1.3B	Oracle	0.88	0.88	0.88	0.88
	In-house LLM-AJ	0.38	0.15	0.38	0.21
	External LLM-AJ	0.63	0.63	0.63	0.63
	CodeBERT	0.81	0.81	0.81	0.81
	CodeT5+	0.80	0.80	0.80	0.79
	Openia	0.81	0.81	0.81	0.81
	AutoProbe	**0.90**	**0.90**	**0.90**	**0.89**
DeepSeek Coder-6.7B	Oracle	0.84	0.84	0.84	0.84
	In-house LLM-AJ	0.48	0.23	0.48	0.32
	External LLM-AJ	0.69	0.70	0.69	0.69
	CodeBERT	0.80	0.80	0.80	0.80
	CodeT5+	0.80	0.81	0.80	0.79
	Openia	0.81	0.81	0.81	0.81
	AutoProbe	**0.82**	**0.82**	**0.82**	**0.82**
Code Llama-7B	Oracle	0.94	0.95	0.94	0.94
	In-house LLM-AJ	0.73	0.53	0.73	0.62
	External LLM-AJ	0.50	0.70	0.50	0.51
	CodeBERT	0.85	0.89	0.85	0.86
	CodeT5+	0.86	0.88	0.86	0.86
	Openia	0.87	0.87	0.87	0.87
	AutoProbe	**0.91**	**0.92**	**0.91**	**0.91**
Code Llama-13B	Oracle	0.89	0.89	0.89	0.89
	In-house LLM-AJ	0.64	0.40	0.64	0.49
	External LLM-AJ	0.54	0.65	0.54	0.53
	CodeBERT	0.82	0.84	0.82	0.83
	CodeT5+	0.83	0.84	0.83	0.83
	Openia	0.75	0.75	0.75	0.75
	AutoProbe	**0.88**	**0.88**	**0.88**	**0.88**
Magicoder-7B	Oracle	0.87	0.87	0.87	0.87
	In-house LLM-AJ	0.51	0.26	0.51	0.35
	External LLM-AJ	0.65	0.66	0.65	0.64
	CodeBERT	0.75	0.76	0.75	0.75
	CodeT5+	0.79	0.79	0.79	0.79
	Openia	0.73	0.73	0.73	0.73
	AutoProbe	**0.85**	**0.85**	**0.85**	**0.85**
Code Gemma-7B	Oracle	0.89	0.89	0.89	0.89
	In-house LLM-AJ	0.51	0.26	0.51	0.35
	External LLM-AJ	0.63	0.65	0.63	0.62
	CodeBERT	0.78	0.78	0.78	0.78
	CodeT5+	0.77	0.78	0.77	0.77
	Openia	0.77	0.79	0.77	0.77
	AutoProbe	**0.88**	**0.88**	**0.88**	**0.88**
GPT-OSS-20B	Oracle	0.89	0.89	0.89	0.89
	In-house LLM-AJ	0.32	0.10	0.32	0.15
	External LLM-AJ	0.37	0.45	0.37	0.38
	CodeBERT	0.76	0.77	0.76	0.76
	CodeT5+	0.84	0.84	0.84	0.84
	Openia	0.75	0.74	0.75	0.75
	AutoProbe	**0.89**	**0.89**	**0.89**	**0.89**

when transitioning from the *independent-unit* setting to a more realistic and complex *repo-level* setting. For example, when evaluating the functional correctness of standalone programs generated by Code Gemma-7B, In-house LLM-AJ and External LLM-AJ achieve F1-Scores of 0.22 and 0.91, respectively. However, these scores significantly drop to 0.05 and 0.80 when assessing the correctness of *repo-level* code. This performance degradation highlights a key limitation of *reasoning-based* methods like LLM-AJ, they rely on provided contextual information to judge code correctness. In the *independent-unit* setting, the program is self-contained, allowing LLMs to access to the full context needed for reliable reasoning. Meanwhile, *repo-level* code involves multi-file dependencies, making it infeasible to capture the complete context within a single prompt. As a result, LLM-AJ lacks the necessary information to reason effectively, leading to degraded performance. These findings suggest that *although* External LLM-AJ *performs well in standalone tasks, it is less reliable in practical scenarios*.

Table 5
Code quality assessment performance in ***functionality*** criterion

Code LLM		Independent-unit code		Repo-level code	
		Accuracy	F1-Score	Accuracy	F1-Score
DeepSeek Coder-1.3B	Oracle	0.66	0.66	0.88	0.85
	In-house LLM-AJ	0.55	0.50	0.24	0.27
	External LLM-AJ	**0.93**	**0.93**	**0.87**	**0.87**
	CodeBERT	0.50	0.45	0.86	0.81
	CodeT5+	0.47	0.34	0.86	0.83
	Openia	0.66	0.66	0.86	0.83
	AutoProbe	0.63	0.63	**0.87**	0.84
DeepSeek Coder-6.7B	Oracle	0.77	0.72	0.77	0.75
	In-house LLM-AJ	0.32	0.26	0.48	0.50
	External LLM-AJ	**0.94**	**0.94**	0.70	0.71
	CodeBERT	0.35	0.35	0.71	0.68
	CodeT5+	0.66	0.66	0.73	0.69
	Openia	0.65	0.67	0.75	**0.73**
	AutoProbe	0.71	0.71	**0.76**	**0.73**
Code Llama-7B	Oracle	0.69	0.69	0.81	0.79
	In-house LLM-AJ	0.61	0.46	0.51	0.54
	External LLM-AJ	**0.97**	**0.97**	0.73	0.74
	CodeBERT	0.58	0.54	0.74	0.69
	CodeT5+	0.63	0.62	0.76	0.71
	Openia	0.69	0.68	**0.81**	**0.79**
	AutoProbe	0.68	0.68	**0.81**	0.78
Code Llama-13B	Oracle	0.74	0.73	0.82	0.79
	In-house LLM-AJ	0.41	0.23	0.19	0.06
	External LLM-AJ	**0.95**	**0.95**	0.77	0.78
	CodeBERT	0.61	0.61	0.80	0.74
	CodeT5+	0.62	0.62	0.78	0.73
	Openia	0.64	0.64	**0.81**	0.78
	AutoProbe	0.68	0.66	**0.81**	**0.79**
Magicoder-7B	Oracle	0.74	0.69	0.78	0.74
	In-house LLM-AJ	0.27	0.11	0.74	0.65
	External LLM-AJ	**0.94**	**0.94**	0.70	0.72
	CodeBERT	0.46	0.49	0.73	0.69
	CodeT5+	0.54	0.56	0.72	0.69
	Openia	0.56	0.58	0.77	0.73
	AutoProbe	0.69	0.66	**0.81**	**0.78**
Code Gemma-7B	Oracle	0.70	0.70	0.84	0.76
	In-house LLM-AJ	0.39	0.22	0.16	0.05
	External LLM-AJ	**0.91**	**0.91**	0.79	**0.80**
	CodeBERT	0.56	0.53	0.82	0.79
	CodeT5+	0.59	0.57	0.79	0.77
	Openia	0.61	0.59	0.82	0.77
	AutoProbe	0.65	0.64	**0.84**	0.77
GPT-OSS-20B	Oracle	0.97	0.96	0.77	0.76
	In-house LLM-AJ	0.87	0.81	0.41	0.36
	External LLM-AJ	**0.97**	**0.97**	0.65	0.66
	CodeBERT	0.88	0.89	0.70	0.67
	CodeT5+	0.92	0.92	0.70	0.67
	Openia	0.96	0.96	**0.74**	0.72
	AutoProbe	0.96	0.95	**0.74**	**0.74**

In contrast, the other approaches shows performance *gains* when moving from *independent-unit* to *repo-level* functionality assessment. For instance, the black-box method with CodeBERT improves by 52%, while AutoProbe achieves a 21% increase. These methods, including black-box classifiers, Openia, and AutoProbe, are *representation-based* approaches that distinguish correct and incorrect code by leveraging features encoded in embedding vectors or hidden states of the generated code. In the *independent-unit* setting, generated code snippets are often short and simple, offering limited discriminative signals. This makes it more challenging for the representation-based models to separate correct and incorrect cases. On the other hand, the *repo-level* setting involves longer and more complex code, which introduces richer structural and semantic information. These characteristics lead to more distinctive feature patterns in the representations, thereby improving separability. As a result, representation-based approaches demonstrate improved

Code Llama

```
Here is a possible implementation of
the `dif_Square` function:
```python
def dif_Square(n):
    i = 1
    while i * i <= n:
        j = 1
        while j * j <= n:
            if i * i + j * j == n:
                return True
            j += 1
        i += 1
    return False
```
This function uses a nested loop
...
additional checks may be needed to
ensure that the function works
correctly.
```

OPENIA: ✓
AUTOPROBE: ✓

DeepSeek Coder

```
Your test case assert dif_Square(15)
...
Here's a Python function that solves
the problem:

```python
def dif_Square(n):
    i = 0
    while i*i <= n:
        j = 0
        while j*j <= n:
            if i*i + j*j == n:
                return True
            j += 1
        i += 1
    return False
```
This function works by
...
If the function does not find such a
pair after checking all pairs, it
returns False.
```

OPENIA: ✗
AUTOPROBE: ✓

MagicCoder

```
Here is the Python function for your
problem:

```python
def dif_Square(n):
    i = 1
    while i * i <= n:
        j = 1
        while j * j <= n:
            if i * i + j * j == n:
                return True
            j += 1
        i += 1
    return False
```

This function works by
...
If it doesn't find such pairs after
checking all possible combinations, it
returns `False`.
```

OPENIA: ✗
AUTOPROBE: ✓

Figure 3: An example of code generated by CODE LLAMA-7B, DEEPSEEK CODER-6.7B, and MAGICODER-7B for Task ID_72 in the MBPP benchmark. The task requires "`Write a Python function to check whether the given number can be represented as difference of two squares or not`". Due to space constraints, portions of the generated outputs are omitted. Full generations are available on our website [56].

performance of functionality assessment when transitioning from the *independent-unit* to the *repo-level* setting.

Furthermore, compared to the SOTA white-box approach OPENIA, AUTOPROBE exhibits greater robustness. Figure 3 shows an example of code generated by CODE LLAMA-7B, DEEPSEEK CODER-6.7B, and MAGICODER-7B for the same task. Although all the three models produced similar implementations, the generated functions are all incorrect. Specifically, instead of checking whether a number can be represented as the *difference* of two squares, the generated code mistakenly checks for the *sum* of two squares.

Despite the nearly identical functions, OPENIA yields inconsistent predictions about their functional correctness. It accurately assesses the correctness of the code generated by CODE LLAMA-7B, but misclassified the outputs from DEEPSEEK CODER-6.7B and MAGICODER-7B. This inconsistency arises because OPENIA relies on the *fixed* representation of only the last generated token (e.g., token `correctly` for CODE LLAMA and `False` for the other models) at the final layer of the models. Since different model architectures and generation dynamics assign varying semantic signals to these tokens, OPENIA's predictions become unstable and unreliable across models, even when the generated code is essentially the same.

Meanwhile, AUTOPROBE overcomes this limitation through an adaptive mechanism that dynamically selects the most informative internal signals for each model. This flexibility allows AUTOPROBE to remain robust to architectural differences and consistently produce correct predictions for the code generated by all three models.

5.1.3. Compilability Assessment

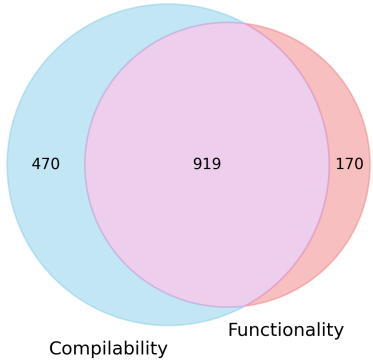
Table 6 shows the results of the compilability assessment under two levels of code generation granularity, *independent-unit* code and *repo-level* code. As seen, *most code quality assessment approaches achieve strong performance in independent-unit code, yet show considerable decline in the more complex repo-level setting*. Among them, AUTOPROBE *demonstrates the highest robustness, maintaining the smallest performance drop between the two settings.*

In the *independent-unit* setting, EXTERNAL LLM-AJ achieves the highest performance, with an average accuracy of 0.92, which is 69% higher than IN-HOUSE LLM-AJ and 11% higher than black-box classification with CodeT5+. However, this strong performance does not generalize to more complex scenarios. When applied to *repo-level* code, EXTERNAL LLM-AJ's accuracy drops significantly to 0.59, representing a 1.5X decrease. Indeed, determining whether code is compilable primarily involves checking for syntax and structural correctness, without the need for deep reasoning. In *independent-unit* snippets, all relevant information, such as imports, variable declarations, and function definitions, is self-contained, allowing LLMs like o4-mini to simply scan the code to verify its syntactic validity. Meanwhile, *repo-level* code often references variables, functions, or modules defined across multiple files. Without explicit access to the entire project context, the LLM struggles to accurately determine compilability, leading to a significant performance drop in this setting.

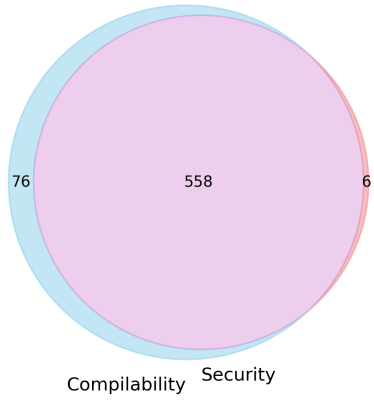
Table 6
Code quality assessment performance in ***compilability*** criterion

Code LLM		Independent-unit code		Repo-level code	
		Accuracy	F1-Score	Accuracy	F1-Score
DeepSeek Coder-1.3B	Oracle	0.90	0.86	0.67	0.67
	In-house LLM-AJ	0.53	0.37	0.48	0.32
	External LLM-AJ	**0.91**	**0.89**	0.55	0.50
	CodeBERT	0.84	0.83	0.61	0.60
	CodeT5+	0.88	0.84	0.61	0.59
	Openia	0.88	0.85	0.60	0.59
	AutoProbe	0.89	0.85	**0.65**	**0.65**
DeepSeek Coder-6.7B	Oracle	0.96	0.94	0.68	0.68
	In-house LLM-AJ	0.76	0.65	0.61	0.46
	External LLM-AJ	0.94	0.94	0.61	0.55
	CodeBERT	0.94	0.93	0.57	0.58
	CodeT5+	**0.96**	0.94	0.58	0.59
	Openia	**0.96**	0.94	0.61	0.61
	AutoProbe	**0.96**	**0.95**	**0.64**	**0.64**
Code Llama-7B	Oracle	0.90	0.86	0.66	0.66
	In-house LLM-AJ	0.39	0.22	0.58	0.42
	External LLM-AJ	**0.91**	**0.88**	0.60	0.55
	CodeBERT	0.87	0.84	0.55	0.55
	CodeT5+	0.56	0.65	0.60	0.59
	Openia	0.88	0.86	0.60	0.61
	AutoProbe	0.90	0.86	**0.66**	**0.66**
Code Llama-13B	Oracle	0.90	0.88	0.70	0.71
	In-house LLM-AJ	0.41	0.23	0.57	0.42
	External LLM-AJ	**0.91**	**0.91**	0.60	0.54
	CodeBERT	0.81	0.79	0.60	0.59
	CodeT5+	0.90	0.88	0.62	0.62
	Openia	0.87	0.87	0.66	0.66
	AutoProbe	0.90	0.87	**0.69**	**0.69**
Magicoder-7B	Oracle	0.96	0.94	0.68	0.69
	In-house LLM-AJ	0.73	0.62	0.60	0.45
	External LLM-AJ	0.95	**0.95**	**0.66**	0.63
	CodeBERT	0.94	0.93	0.58	0.58
	CodeT5+	0.88	0.90	0.57	0.57
	Openia	**0.96**	0.94	0.61	0.61
	AutoProbe	**0.96**	0.94	**0.66**	**0.67**
Code Gemma-7B	Oracle	0.88	0.86	0.68	0.68
	In-house LLM-AJ	0.44	0.27	0.52	0.36
	External LLM-AJ	0.88	0.86	0.55	0.49
	CodeBERT	0.81	0.82	0.58	0.56
	CodeT5+	0.78	0.80	0.61	0.60
	Openia	0.84	0.79	0.67	0.67
	AutoProbe	**0.90**	**0.89**	**0.69**	**0.69**
GPT-OSS-20B	Oracle	0.99	0.99	0.76	0.75
	In-house LLM-AJ	0.89	0.84	0.62	0.57
	External LLM-AJ	0.89	0.87	0.67	0.65
	CodeBERT	0.91	0.92	0.65	0.65
	CodeT5+	0.96	0.96	0.64	0.64
	Openia	0.98	0.98	0.69	0.69
	AutoProbe	**0.99**	**0.99**	**0.71**	**0.70**

In the more complex setting of *repo-level* code assessment, AutoProbe achieves the highest performance, outperforming External LLM-AJ by 12% on average. For example, when evaluating the compilability of code generated by Code Gemma-7B, both AutoProbe and External LLM-AJ reach the accuracy of 0.90 on *independent-unit* code. However, for *repo-level* code, AutoProbe obtains an accuracy of 0.69, which is 27% higher than that of External LLM-AJ. Interestingly, in this case, AutoProbe even slightly surpasses the performance of Oracle. These results demonstrate that AutoProbe is less sensitive to the increased code complexity, making it a more reliable choice for practical scenarios.



(a) Correlation of accurate predictions for compilability and functionality



(b) Correlation of accurate prediction for compilability and security

Figure 4: Correlation of accurate predictions across code quality criteria

5.1.4. *Correlation of Accurate Predictions Across Code Quality Criteria*

Figure 4 illustrates the extent to which AutoProbe correctly predicts multiple aspects of code quality simultaneously. This analysis is conducted on code generated by Code Gemma-7B. The results show that AutoProbe *is able to capture the underlying relationship between code quality dimensions, yielding substantial overlap in accurate predictions across criteria.*

For *compilability* and *functionality*, AutoProbe correctly predicts both properties for about 84% of functionally correct code. However, approximately 34% of compilable code is predicted as functionality incorrect, highlighting that structural correctness alone is not a sufficient indicator of behavioral correctness. Meanwhile, only 17% of functionally correct code is misclassified as non-compilable. These figures indicate that compilability is often a prerequisite for functionality and that AutoProbe 's predictions partially reflect this dependency. The overlap between accurate predictions of *compilability* and *security* is substantially stronger. AutoProbe correctly predicts both criteria for nearly 99% of secure samples, with only a negligible number (6 samples) being predicted as secure but non-compilable. This suggests that AutoProbe's security predictions are closely aligned with structural validity.

Overall, these findings demonstrate that AutoProbe not only performs well on individual quality dimensions but also captures their co-occurrence patterns, reflecting the relational structure between code quality dimensions. This supports the idea that code quality assessment can benefit from being framed as a multi-aspect prediction task. As future work, we plan to explore joint modeling strategies that explicitly integrate these quality dimensions, with the goal of improving overall prediction consistency and revealing deeper interactions between structural, functional, and security-related properties.

5.1.5. *Potential in Improving LLM-generated Code's Quality*

To demonstrate the potential of AutoProbe in proactively detecting and mitigating low-quality code before it reaches developers, we introduce a *Quality-Guided Code Generation (QG^2)* pipeline. For each programming task, an LLM generates multiple candidate solutions. After that, AutoProbe assigns a quality score to each candidate and selects the highest-scoring one to present to developers. We evaluate the effectiveness of QG^2 by comparing it against the standard generation process using the *pass@1* metric [1]. Note that, because AutoProbe extracts internal states during generation and feeds them to a lightweight probing classifier with only two hidden layers, the additional time overhead is marginal (more details in Sec. 5.4).

Table 7 shows the pass@1 performance of the studied LLMs with and without QG^2 across code quality dimensions. Overall, *QG^2 can improve the performance of most models across code quality dimensions, with particularly pronounced gains in security*. Specifically, QG^2 increases the proportion of secure code by 11–40% compared to standard generation. Interestingly, this improvement enables the small open-source models to outperform the latest commercial model GPT-5.1 in security-oriented code generation. For example, while GPT-5.1 achieves 56.25% pass@1 on security tasks, about 20% higher than Code Gemma-7B under standard generation, QG^2 enhances Code Gemma-7B's performance to surpass GPT-5.1 by 17%.

For functionality and compilability, QG^2 also yields measurable gains for several models, like DeepSeek Coder-1.3B, Code Gemma-7B, and GPT-OSS, demonstrating its ability to enhance structural and functional correctness. However, its effectiveness is not consistent across all models. For Code Llama, performance with and without QG^2 remain similar on these criteria. This is likely because Code Llama tends to generate highly similar candidate solutions for each task in the evaluation benchmark, offering limited diversity for the ranking mechanism to leverage. In such cases, the selection step has little room to identify a better candidate, diminishing the advantage of QG^2.

Overall, these findings illustrate that integrating AutoProbe into the generation pipeline can enhance the quality

Table 7
Performance of Code LLMs when applying *Quality-Guided Code Generation (QG²)* using AutoProbe (pass@1)

	Security		Functionality		Compilability	
	Without QG²	With QG²	Without QG²	With QG²	Without QG²	With QG²
DeepSeek Coder-1.3B	42.19%	**53.13%**	53.66%	**57.32%**	87.80%	**92.07%**
DeepSeek Coder-6.7B	50.00%	**60.94%**	75.00%	75.00%	75.00%	75.00%
Code Llama-7B	70.31%	**92.19%**	40.24%	40.24%	89.63%	89.63%
Code Llama-13B	64.06%	**84.38%**	55.49%	**61.59%**	90.24%	90.24%
Code Gemma-7B	46.88%	**65.63%**	37.80%	**45.73%**	74.39%	**92.68%**
Magicoder-7B	54.69%	**60.94%**	73.78%	**74.39%**	95.12%	**95.73%**
GPT-OSS-20B	62.50%	**96.88%**	88.41%	**92.07%**	90.85%	**96.34%**
GPT5.1	56.25%		98.17%		100%	

of LLM-generated code across multiple dimensions. Notably, it enables open-source models to compete with or even outperform state-of-the-art commercial systems in critical areas such as security-oriented code generation.

Answer to RQ1: Across all studied LLMs and benchmarks, AutoProbe consistently achieves strong performance on all three code quality dimensions, security, functionality, and compilability. While External LLM-AJ achieves the highest accuracy in the independent-unit setting, its performance substantially drops in more realistic and complex scenarios such as repo-level code, indicating its limited generalizability.

Compared to the SOTA white-box approach Openia, AutoProbe also shows advantages. While Openia can reach performance nearly close to AutoProbe in certain cases, such as functionality assessment on *repo-level* code, its behavior is inconsistent across assessment criteria and model architectures. For instance, Openia significantly lags behind AutoProbe in security assessment and in compilability prediction for *repo-level* setting, reflecting the sensitivity of its fixed state-selection strategy.

In contrast, AutoProbe demonstrates robust performance across experimental settings and consistently outperforms other baselines. This robustness across heterogeneous models and tasks empirically demonstrates the broad applicability and reliability of AutoProbe.

5.2. Intrinsic Analysis

5.2.1. Impact of Representation Sampling Strategies

Token sampling: Figure 5 shows the impact of different token sampling strategies on AutoProbe's performance. Overall, *the boundary-aware strategy offers the best balance, achieving the highest F1-Score with relatively low memory consumption.* While random token sampling results in the lowest memory usage (about 6.5 GB across GPU and CPU), it yields the weakest performance. This is expected because randomly selecting a single token may overlook important signals, leading to suboptimal predictions.

In contrast, AutoProbe with boundary-aware token sampling consumes 2.5 times more memory than random sampling, but achieves significantly better performance. Specifically, AutoProbe reaches an F1-Score of 0.76, outperforming full token and random token sampling strategies by 9% and 26%, respectively. Moreover, compared to full token sampling, the boundary-aware strategy reduces memory usage by 18X and training time by over 100X. These results suggest that using all tokens introduces substantial

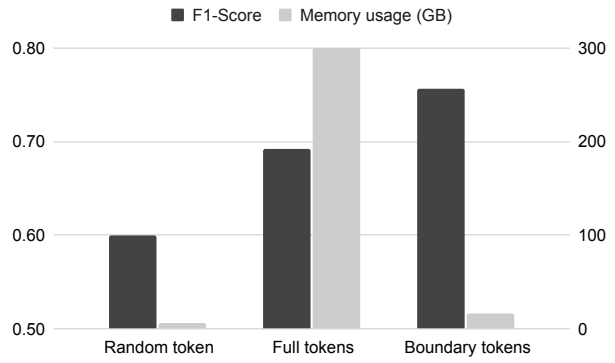


Figure 5: Impact of token sampling on AutoProbe's performance and memory usage, left axis: *F1-Score*; right axis: *Memory usage (GB)*

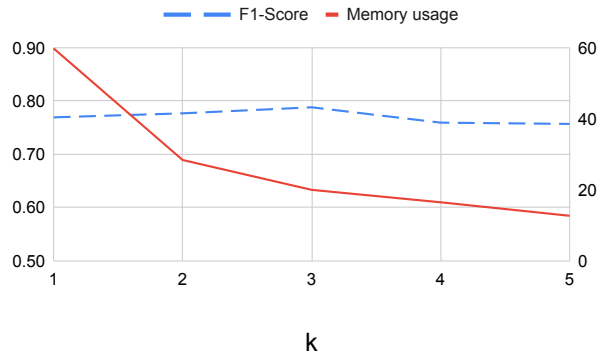


Figure 6: Impact of layer sampling intervals k on AutoProbe's performance and memory usage, left axis: *F1-Score*; right axis: *Memory usage (GB)*

Table 8
Impact of *Informative Representation Selector* on AutoProbe's Performance

	Accuracy	Precision	Recall	F1-Score
Without selector	0.70	0.70	0.70	0.69
With selector	**0.76**	**0.76**	**0.76**	**0.76**

computational overhead and even degrades performance due to the inclusion of noisy and irrelevant representations. Meanwhile, the boundary-aware strategy effectively reduces input size while preserving the most informative signals for code quality assessment.

Layer Sampling: Figure 6 shows how the layer sampling interval $k \in \{1, 2, 3, 4, 5\}$, i.e., how frequently layers are selected for extracting internal states, affects AutoProbe's performance and memory usage. *Across different sampling intervals,* AutoProbe*'s F1-Scores are relatively stable*, fluctuating between 0.76 and 0.79. This stability can be explained by the fact that adjacent layers in transformer-based models tend to encode highly similar features, with similarity increasing as layers get closer [61]. As a result, densely sampling layers (small k) often captures redundant features without providing additional information for code quality prediction.

Meanwhile, *memory usage decreases steadily as k increases,* since larger intervals result in fewer selected layers and a smaller set of extracted representations $\mathcal{H}^*$, thereby lowering memory consumption. For example, when $k = 1$, AutoProbe requires about 60 GB of GPU and CPU, which is 4.7X more than when $k = 5$. These results highlight the importance of selecting a representative subset of layers rather than using all of them, which increases overhead without improving code quality prediction performance.

5.2.2. Impact of Informative Representation Selector

Table 8 illustrates that *the informative representation selector module significantly contributes to* AutoProbe*'s performance*. Indeed, not all internal representations contribute equally to code quality prediction. Without a mechanism to identify and prioritize useful signals, the classifier struggles to capture important information. By incorporating an attention mechanism to assign higher weights to the important representations and downweight the irrelevant ones, AutoProbe's F1-Score in code quality prediction improves from 0.69 to 0.76 (more than 10% relatively). The selector module thus plays a crucial role in directing the model's focus toward the most relevant internal states, leading to more effective and reliable code quality assessment. Moreover, since the attention scores are jointly trained with the probing classifier, the training time remains comparable with and without the selector module. This suggests that the performance improvement comes at a reasonable computational cost, making the selector a lightweight yet useful addition to the AutoProbe's design.

Table 9
Impacts of *Aggregation Functions* on AutoProbe's Performance

Aggregation	Accuracy	Precision	Recall	F1-Score
Concatenation	0.76	0.76	0.76	0.76
Summation	0.76	0.76	0.76	0.76
Mean pooling	0.76	0.76	0.76	0.76
Max pooling	**0.79**	**0.80**	**0.79**	**0.79**
Min pooling	0.73	0.73	0.73	0.73

Table 10
Impact of *Probing Classifiers* on AutoProbe's Performance

Classifier	Accuracy	Precision	Recall	F1-Score
Logistic Regression	**0.77**	**0.77**	**0.77**	**0.77**
MLP	0.76	0.76	0.76	0.76
SVM	0.47	0.22	0.47	0.30

5.2.3. Impact of Aggregation and Probing Classier

Table 9 presents the performance of AutoProbe with different ***aggregation functions***, which are applied to combine the weighted representations in $\mathcal{Z}^*$ before feeding them to a probing classifier (Sec. 3.4). Among the evaluated strategies, *max pooling proves to be the most effective aggregation function, enabling* AutoProbe *to achieve the highest prediction performance.* As seen, with *max pooling*, AutoProbe achieves an accuracy of 0.79, which is about 10% higher than the worst setting using *min pooling*. This indicates that *max pooling*, by selecting the most prominent features across representation dimensions, captures more discriminative signals for code quality prediction, thereby enhancing AutoProbe's overall performance.

The performance of different ***probing classifiers*** used in AutoProbe is shown in Table 10. *Both Logistic Regression and MLP achieve comparable results, while the SVM classifier performs significantly worse*. Specifically, AutoProbe obtains an average accuracy of about 0.77 with either Logistic Regression or MLP, whereas this figure drops to only 0.47 when using SVM. SVM is not a reliable choice in this context due to its limited capability in handling high-dimensional and possibly noisy feature spaces, which is an inherent characteristic of the internal representations extracted from LLMs. Furthermore, SVM is not typically suitable for joint training with upstream components such as the attention module, as their standard optimization is not gradient-based. Although SVM can be trained using gradient-based methods via hinge loss, the loss function itself is non-smooth, which can lead to unstable or inefficient gradient propagation. In contrast, Logistic Regression and MLP are trained using gradient descent and are fully differentiable, making them naturally compatible with AutoProbe's end-to-end learning framework.

Answer to RQ2: Each component of AutoProbe effectively contributes to its overall performance and

Table 11
Code quality assessment performance in F1-score of the approaches across different programming languages

Targeted language	CodeBERT	CodeT5+	OPENIA	AUTOPROBE
CPP	0.65	0.67	**0.82**	0.76
C Sharp	0.74	0.75	0.80	**0.85**
Java	0.59	0.64	0.68	**0.76**
JavaScript	0.66	0.80	0.86	**0.89**
PHP	0.54	0.65	0.71	**0.83**
Python	0.37	0.50	0.67	**0.78**
Shell Script	0.58	0.58	0.64	**0.67**
TypeScript	0.72	0.82	**0.88**	0.86
Average	0.61	0.68	0.76	**0.80**

robustness. Within the internal representation extractor, compact and structured subsets of internal states, which are obtained through suitable token and layer sampling strategies, significantly improve efficiency without sacrificing predictive accuracy. The informative representation selector and the max pooling aggregation function further enhance performance by capturing the most salient features from the selected representations. Finally, gradient-based classifiers such as Logistic Regression and MLP are well-suited for integration into AUTOPROBE's joint training framework.

5.3. Sensitivity Analysis

5.3.1. Impact of Programming Language Variability

Table 11 shows the generalization performance of the approaches in assessing the code quality across different programming languages. This experiment is conducted on the multilingual version of *HumanEval* benchmark. For each task in a target language, DEEPSEEK CODER-6.7B is employed to generate 10 candidate solutions. To evaluate cross-language generalization, one programming language is held out for testing, while the classifiers are trained on either the generated code (for post-hoc classifications) or internal representations (for OPENIA and AUTOPROBE) from the remaining languages.

Overall, AUTOPROBE *exhibits strong generalization capability across diverse programming languages*. AUTOPROBE achieves the highest average F1-Score of 0.80, outperforming OPENIA by 5% and the black-box methods by up to 32%. Notably, for Python programs, AUTOPROBE obtains an F1-score of 0.78, improving over the baselines by margins ranging from 17% to 111%.

Moreover, *white-box approaches, such as* AUTOPROBE *and* OPENIA, *demonstrate stability across languages*, maintaining their performance within a relatively narrow F1-score range. In contrast, black-box models, like CodeBERT and CodeT5+, show greater variability, with performance dropping significantly for certain languages. For example, while CodeT5+ performs competitively on TypeScript (F1-score of 0.82), its performance declines sharply to just 0.50 on Python. The reason is that black-box approaches operate on final code outputs; thus, they could be sensitive to language-specific characteristics such as keywords, syntax, and conventions. Meanwhile, white-box approaches leverage LLMs' internal representations, allowing them to capture deeper semantic understanding rather than relying on shallow surface-level features such as token sequences. As a result, the white-box approaches, like AUTOPROBE or OPENIA, can exhibit greater robustness across languages and programming paradigms.

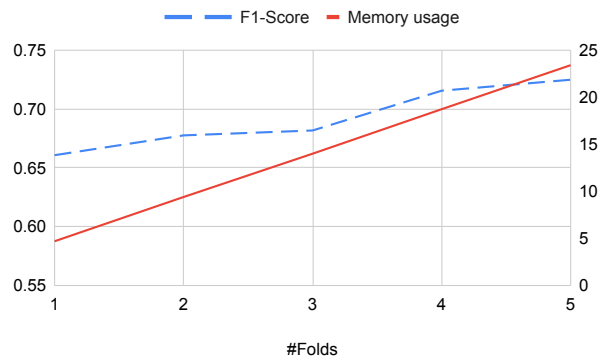


Figure 7: Impact of training size on AUTOPROBE's performance and memory usage, left axis: *F1-Score*; right axis: *Memory Usage (GB)*

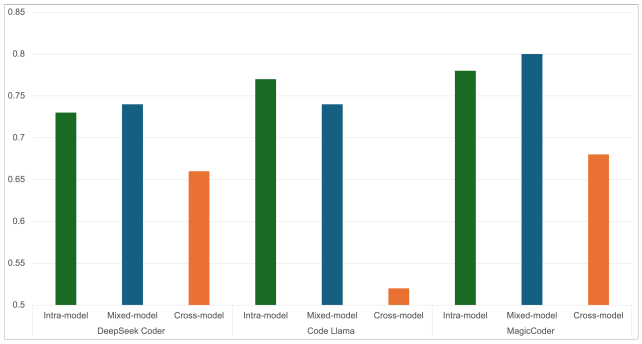


Figure 8: Impact of LLM generalization between training and inference on AUTOPROBE's performance on F1-Score

5.3.2. Impact of Training Data Size

Figure 7 shows that *increasing the training data size enhances classification accuracy, but also leads to higher memory consumption.* Specifically, as the training size increases from 1 fold to 5 folds, the F1-score steadily improves from 0.66 to 0.72. This suggests that a larger training set enables the probing classifier and attention mechanism to better learn and generalize code quality-related patterns. Moreover, memory consumption grows almost linearly with the training size, ranging from around 5 GB at 1 fold to about 23 GB at 5 folds of data. This is expected, as more training examples lead to larger sets of internal representations being processed and stored during training. The results emphasize the importance of selecting a training size that balances performance needs with computational resources.

5.3.3. Model Generalization Across Code LLMs

Figure 8 shows AUTOPROBE's performance when trained on internal states from either the same or different models

as the test-time model. This experiment employs three code LLMs, DeepSeek Coder-6.7B, Code Llama-7B, and Magicoder-7B, which share a common architecture of 32 layers and 4,096-dimensional hidden states. We evaluates three settings: *intra-model*, *cross-model*, and *mixed-model*. In the *intra-model* setting, the classifier is both trained and tested on the internal states of the same model. In the *cross-model* setting, training is conducted on the internal states of two LLMs, while testing is performed on the remaining one. In the *mixed-model* setting, the classifier is trained on the combined internal states of all three models and evaluated individually on each model.

Across all three models, AutoProbe *performs worst in the cross-model setting*. For example, when evaluating on Code Llama, AutoProbe achieves an F1-Score of 0.77 in the *intra-model* setting, but its performance drops to 0.52 in the *cross-model* setting. A similar trend is observed for Magicoder and DeepSeek Coder, where AutoProbe's F1-Scores in where F1-scores decline from 0.78 and 0.73 (*intra-model*) to 0.68 and 0.66 (*cross-model*), respectively.

AutoProbe*'s performance in the mixed-model setting varies across models*. For Magicoder and DeepSeek Coder, AutoProbe's performance in the *mixed-model* setting achieves a slightly higher F1-score than that in the *intra-model* setting. This result suggests that incorporating representations from multiple models can enrich the training data for the classifier. However, for Code Llama, in the *mixed-model* setting, AutoProbe performs worse than when in the *intra-model* setting. This also indicates that the inclusion of external representations may introduce noise. Overall, these results highlight that AutoProbe's generalization performance depends on the degree of alignment among representational spaces of different LLMs and on the classifier's ability to extract shared quality-related features across them.

These results show that while AutoProbe can effectively adapt to different models, it is not entirely independent of model architectures in a zero-shot transfer setting. In practice, the probing classifiers still benefit from the representational distributions of the model they are trained on, an expected characteristic of supervised probing approaches. Because architectural design, training dynamics, and pretraining data vary across LLMs, the distributions of correctness-related signals are not guaranteed to transfer directly between models. These findings underscore the importance of aligning the training and inference models to ensure optimal performance in white-box code quality assessment.

Answer to RQ3: AutoProbe generalizes well across programming languages and data scales, with optimal performance achieved when training and inference use internal representations from the same code LLM or mixture of similar models.

5.4. Efficiency Analysis

The time complexity of AutoProbe is primarily influenced by three components: (1) extracting internal representations, (2) training the attention scores and probing classifier, and (3) performing inference. Internal representations were extracted from the studied code LLMs on an NVIDIA A6000 GPU with 48GB of VRAM. This process required approximately 125 hours for the entire dataset. Importantly, the extraction was performed concurrently during code generation and did not introduce additional latency into the generation process.

All subsequent experiments, including training and inference of the probing classifier, were conducted on an Ubuntu Server 22.04 equipped with a single NVIDIA A6000 GPU (48GB VRAM) and 512GB of system RAM. Since AutoProbe bypasses costly embedding computations, its classifier training and inference are extremely fast. Specifically, the training phase requires just about 1 minute, and the inference phase takes only 0.6 milliseconds per generated code unit. Meanwhile, the black-box baselines such as CodeBERT and CodeT5+ are significantly more time-consuming, taking around 83 minutes to train, with over 98% of the time spent on embedding generation.

Regarding memory usage, the standard variant of AutoProbe consumes about 16.5 GB of GPU and CPU memory. However, When token position sampling is disabled, i.e., using the representations of all generated tokens, AutoProbe's memory consumption increases substantially, reaching up to 300 GB.

Answer to RQ4: AutoProbe achieves high efficiency in both training and inference. By employing token and layer sampling strategies it significantly reduces memory consumption without sacrificing performance. This makes AutoProbe a practical and scalable solution for real-world deployment.

5.5. Threats to Validity

The main threats to the validity of our work consist of internal, construct, and external threats.

Internal Validity. One threat to internal validity lies in the selection of hyperparameters. To mitigate this threat, we systematically explored various configurations of AutoProbe and reused the official settings from the original papers for baselines [28, 26, 27], ensuring fair comparisons. Another potential threat involves the procedure for extracting internal representations. To address this threat, we relied on widely adopted PyTorch tools to extract hidden states during code generation. Additionally, to further reduce the risk of implementation errors, we carefully reviewed our code and made it public [56] so that other researchers can double-check and reproduce our experiments.

Construct Validity. A threat to construct validity arises from the selection of evaluation metrics and experimental procedures. To mitigate this threat, we adopt standard metrics, including Accuracy, Precision, Recall, and F1-Score for evaluation. To ensure a comprehensive assessment, we systematically evaluated AutoProbe across multiple code

LLMs, benchmarks, and experimental settings. Another potential threat involves the procedure for labeling code quality, which relies on test outcomes or static analysis results. To mitigate this threat, we utilized high-quality, widely adopted benchmarks [1, 10, 12, 50, 51, 52, 53] that include rigorous test suites. As future work, we plan to incorporate human evaluation to complement automated assessments.

External Validity. External threat relates to the generalizability of our results. We mitigated this threat by evaluating AutoProbe across diverse model families. However, due to hardware constraints, our evaluation focused on models with up to 13 billion parameters. We intend to explore the scalability of AutoProbe to larger models in future work. Additionally, the prompt variability also poses a potential threat. To address this, we conducted evaluations on diverse benchmarks with varying prompt styles, offering partial mitigation. We also plan to systematically study prompt sensitivity in future experiments.

6. Related Work

LLM-based code generation. LLMs, known for their powerful capabilities in contextual understanding and response generation, have been widely applied across various domains, including natural language processing [39, 40, 62, 63], vision-language integration [64, 65], and software engineering (SE) tasks [4, 42, 43, 3, 2]. In the context of SE, code LLMs, such as DeepSeek Coder [8], Code Llama [9], and Magicoder [41], which have been pre-trained or fine-tuned on large-scale code corpora, have demonstrated remarkable success in a wide range of tasks, e.g. code generation [2, 3, 66], test generation [43, 67], code summarization [68], and program repair [69], etc. These models are increasingly integrated into modern development environments via tools like GitHub Copilot [6] and CodeGeeX2 [7], where they assist developers in generating boilerplate code, completing functions, or writing test cases. Despite their impressive performance, a key challenge remains, the hallucination problem [44, 70, 11, 71], where LLMs generate code that is syntactically valid but semantically incorrect, insecure, or unexecutable. Such hallucinations compromise the reliability of LLM-generated code and raise concerns about correctness, security, and trustworthiness. Mitigating such issues remains an open problem and is essential for ensuring the reliable and effective adoption of code LLMs in real-world software projects.

LLM's hallucination detection. Various approaches [45, 33, 30, 72, 73] have been proposed to detect hallucinations in LLMs. These approaches can be broadly categorized into black-box and white-box approaches, depending on whether they rely on the model's input-output behaviors or leverage its internal computations.

Black-box approaches [72, 74, 75, 76, 77] detect hallucinations by relying solely on the input-output behavior of the models. A common strategy is *uncertainty estimation*, based on a hypothesis that responses with higher model confidence are more likely to be correct [72, 74, 78]. For instance, SelfCheckGPT [72] estimates model confidence by sampling multiple responses to the same query and measuring their semantic consistency. If the responses are consistent across samples, the output is considered to be reliable; otherwise, it is likely hallucinated. Another research direction focuses on *fact-checking*, where the generated outputs are validated against factual sources. These sources can be drawn from the model's internal knowledge [75] or external corpora [76, 79, 80]. Additionally, some other black-box methods formulate hallucination detection as a *classification task*, training a downstream classifier on features extracted from the generated outputs to distinguish hallucinated from faithful outputs [77].

On the other hand, ***white-box*** approaches leverage the model's internal computations, such as *hidden states*, *activations*, *attention maps*, to determine unreliable or hallucinated responses [30, 31]. For example, Azaria *et al.* [32] train a classifier on the hidden activations to estimate the truthfulness of the generated statements. Similarly, INSIDE [33] and FactoScope [81] analyze internal states to assess semantic consistency and factual reliability.

Our work follows the while-box paradigm by leveraging internal representations for hallucination detection. However, unlike prior studies [29, 33, 45, 32] that focus on unstructured Natural Language Generation (NLG) tasks, where correctness is often subjective by relying on human judgment or based on external factual knowledge, AutoProbe target *code correctness*, where correctness is objectively defined by syntax, semantics, and functional behaviors. Due to the nature of NLG tasks, previous works [33, 32] mainly evaluate hallucination through semantic consistency or factual alignment among model responses. However, these techniques are not directly applicable to code. In the context of code generation, a task may have multiple correct solutions that differ in implementation details or algorithms. As a result, inconsistency across generated responses does not necessarily imply incorrectness. To address this gap, AutoProbe is specifically designed for assessing code quality using internal representations.

Code quality assessment. Ensuring code quality is a fundamental objective in the SE. Traditional approaches for detecting program issues often rely heavily on handcrafted features and static analysis techniques [82]. With the emergence of deep learning, pre-trained models such as CodeBERT [26], GraphBERT [83], and CodeT5 [27] have been widely adopted due to their ability to capture rich contextual and semantic information from source code. These models support automatic feature extraction from different code representations such as raw code, program slices, or code property graphs, enabling bug and vulnerability detection across different levels of granularity, from coarse-grained levels, i.e., files or methods, to fine-grained [84, 85, 86, 87, 88], i.e., slices and statements. For instance, IVDetect [84] employs a graph-based neural network to detect vulnerabilities at the function level and uses interpretable models to further localize vulnerable statements. LineVul [88] and LineVD [87] leverage CodeBERT to encode code representations and

have demonstrated superior performance over IVDetect in fine-grained vulnerability detection.

Recently, the quality of code generated by LLMs has gained significant attention due to the increasing reliance on AI-generated code in real-world applications. Several empirical studies have revealed the potential risks of bugs and security issues associated with LLM-generated code [89, 90, 91, 92, 93]. Multiple studies [78, 94, 95] have been proposed for guaranteeing the quality of code generated by LLMs. Huang *et al.* [78] propose a technique of uncertainty analysis to measure LLMs' confidence in generated outputs, which could aid in identifying potentially unreliable code. Furthermore, multi-agent frameworks such as AutoSafeCoder [95] enhance LLM-based code generation by integrating static analysis and fuzz testing. Similarly, LLMSecGuard [94] combines the capabilities of static analyzers and LLMs to improve code security, demonstrating the benefits of hybrid approaches in mitigating vulnerabilities and strengthening code robustness.

Openia [28] demonstrates that the internal states of LLMs encode valuable signals related to code correctness and can be leveraged to detect incorrect code. This work is closely related to ours, as both adopt a white-box approach. However, the key difference between AutoProbe and Openia lies in the mechanism for selecting informative internal representations. Openia assumes that the hidden states of the last generated tokens at the last layer encapsulate the most comprehensive information and therefore uses them directly for correctness assessment. Instead of selecting a fixed representation that could lead to suboptimal performance across models, AutoProbe proposes a model-agnostic approach that dynamically selects the most informative representations across multiple layers and token positions. This flexibility enables AutoProbe to better adapt to different model architectures.

7. Conclusion

In this paper, we introduced AutoProbe, a model-agnostic framework for assessing the quality of LLM-generated code by dynamically selecting and leveraging informative internal representations. Specifically, AutoProbe uses an attention-based mechanism to learn which internal states are most predictive of code quality, enabling better adaptation across diverse LLM architectures and tasks. Our experimental results demonstrate that AutoProbe consistently outperforms SOTA black-box and white-box baselines across different quality criteria, including compilability, functionality, and security. These findings highlight that dynamically selecting important internal signals enables AutoProbe to serve as a robust and generalizable solution for assessing the quality of code generated by various LLMs.

Acknowledgement

This research was funded by VNU Asia Research Center (ARC) from grants source by CHEY Institute for Advanced Studies, code CA.24.13A.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374.

[2] T.-D. Bui, D.-T. Luu-Van, T.-P. Nguyen, T.-T. Nguyen, S. Nguyen, H. D. Vo, Rambo: Enhancing rag-based repository-level method body completion, arXiv preprint arXiv:2409.15204.

[3] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, W. Chen, Repocoder: Repository-level code completion through iterative retrieval and generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 2471–2484.

[4] Z. Zheng, K. Ning, Q. Zhong, J. Chen, W. Chen, L. Guo, W. Wang, Y. Wang, Towards an understanding of large language models in software engineering tasks, Empirical Software Engineering 30 (2) (2025) 50.

[5] S.-C. Dai, J. Xu, G. Tao, A comprehensive study of llm secure code generation, arXiv preprint arXiv:2503.15554.

[6] Github copilot.
URL https://github.com/features/copilot

[7] Codegeex2.
URL https://github.com/zai-org/CodeGeeX2

[8] DeepSeek, Deepseek coder: Let the code write itself, https://github.com/deepseek-ai/DeepSeek-Coder (2023).

[9] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al., Code llama: Open foundation models for code, arXiv preprint arXiv:2308.12950.

[10] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al., Program synthesis with large language models, arXiv preprint arXiv:2108.07732.

[11] J. Liu, C. S. Xia, Y. Wang, L. Zhang, Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, Advances in Neural Information Processing Systems 36 (2023) 21558–21572.

[12] J. Li, G. Li, Y. Zhao, Y. Li, H. Liu, H. Zhu, L. Wang, K. Liu, Z. Fang, L. Wang, J. Ding, X. Zhang, Y. Zhu, Y. Dong, Z. Jin, B. Li, F. Huang, Y. Li, B. Gu, M. Yang, DevEval: A manually-annotated code generation benchmark aligned with real-world code repositories, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 3603–3614. doi:10.18653/v1/2024.findings-acl.214.
URL https://aclanthology.org/2024.findings-acl.214/

[13] N. Perry, M. Srivastava, D. Kumar, D. Boneh, Do users write more insecure code with ai assistants?, in: Proceedings of the 2023 ACM SIGSAC conference on computer and communications security, 2023, pp. 2785–2799.

[14] Z. Liu, Y. Tang, X. Luo, Y. Zhou, L. F. Zhang, No need to lift a finger anymore? assessing the quality of code generation by chatgpt, IEEE Transactions on Software Engineering 50 (6) (2024) 1548–1584.

[15] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, L. C. Cordeiro, How secure is ai-generated code: A large-scale comparison of large language models, Empirical Software Engineering 30 (2) (2025) 47.

[16] K. Jesse, T. Ahmed, P. T. Devanbu, E. Morgan, Large language models and simple, stupid bugs, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR), IEEE, 2023, pp. 563–575.

[17] F. Tambon, A. M. Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, G. Antoniol, Bugs in large language models generated code, arXiv preprint arXiv:2403.08937.

[18] Z. Liu, Y. Tang, X. Luo, Y. Zhou, L. F. Zhang, No need to lift a finger anymore? assessing the quality of code generation by chatgpt, IEEE Transactions on Software Engineering.

[19] J. Liu, C. S. Xia, Y. Wang, L. Zhang, Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, Advances in Neural Information Processing Systems 36.

[20] H. Krasner, The cost of poor software quality in the us: a 2022 report, Consortium for Information and Software Quality (CISQ).

[21] J. T. Liang, C. Yang, B. A. Myers, A large-scale survey on the usability of ai programming assistants: Successes and challenges, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 1–13.

[22] P. Vaithilingam, T. Zhang, E. L. Glassman, Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models, in: Chi conference on human factors in computing systems extended abstracts, 2022, pp. 1–7.

[23] Z. Ding, H. Li, W. Shang, T.-H. P. Chen, Can pre-trained code embeddings improve model performance? revisiting the use of code embeddings in software engineering tasks, Empirical Software Engineering 27 (3) (2022) 1–38.

[24] Y. Zhao, L. Gong, Z. Huang, Y. Wang, M. Wei, F. Wu, Coding-ptms: How to find optimal code pre-trained models for code embedding in vulnerability detection?, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1732–1744.

[25] C. Niu, C. Li, B. Luo, V. Ng, Deep learning meets software engineering: A survey on pre-trained models of source code, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22, 2022, pp. 5546–5555.

[26] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 1536–1547.

[27] Y. Wang, W. Wang, S. Joty, S. C. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021, pp. 8696–8708.

[28] T.-D. Bui, T. T. Vu, T.-T. Nguyen, S. Nguyen, H. D. Vo, Correctness assessment of code generated by large language models using internal representations, Journal of Systems and Software 230 (2025) 112570. doi:https://doi.org/10.1016/j.jss.2025.112570.

[29] Z. Ji, D. Chen, E. Ishii, S. Cahyawijaya, Y. Bang, B. Wilie, P. Fung, Llm internal states reveal hallucination risk faced with a query, in: Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP, 2024, pp. 88–104.

[30] J. Ferrando, G. Sarti, A. Bisazza, M. R. Costa-jussà, A primer on the inner workings of transformer-based language models, arXiv preprint arXiv:2405.00208.

[31] S. Zhao, T. Nguyen, A. Grover, Probing the decision boundaries of in-context learning in large language models, Advances in Neural Information Processing Systems 37 (2024) 130408–130432.

[32] A. Azaria, T. Mitchell, The internal state of an llm knows when it's lying, in: Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 967–976.

[33] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, J. Ye, Inside: Llms' internal states retain the power of hallucination detection, arXiv preprint arXiv:2402.03744.

[34] L. Floridi, M. Chiriatti, Gpt-3: Its nature, scope, limits, and consequences, Minds and machines 30 (4) (2020) 681–694.

[35] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, et al., Palm 2 technical report, arXiv preprint arXiv:2305.10403.

[36] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971.

[37] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, J. Gao, Large language models: A survey, arXiv preprint arXiv:2402.06196.

[38] B. Ji, X. Duan, Y. Zhang, K. Wu, M. Zhang, Zero-shot prompting for llm-based machine translation using in-domain target sentences, IEEE/ACM Transactions on Audio, Speech, and Language Processing.

[39] H. Zhang, P. S. Yu, J. Zhang, A systematic survey of text summarization: From statistical methods to large language models, ACM Computing Surveys 57 (11) (2025) 1–41.

[40] Z. Li, S. Fan, Y. Gu, X. Li, Z. Duan, B. Dong, N. Liu, J. Wang, Flexkbqa: A flexible llm-powered framework for few-shot knowledge base question answering, in: Proceedings of the AAAI conference on artificial intelligence, Vol. 38, 2024, pp. 18608–18616.

[41] Y. Wei, Z. Wang, J. Liu, Y. Ding, L. Zhang, Magicoder: Source code is all you need, arXiv preprint arXiv:2312.02120.

[42] R. Wang, J. Guo, C. Gao, G. Fan, C. Y. Chong, X. Xia, Can llms replace human evaluators? an empirical study of llm-as-a-judge in software engineering, Proceedings of the ACM on Software Engineering 2 (ISSTA) (2025) 1955–1977.

[43] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, J. Yin, Chatunitest: A framework for llm-based test generation, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, 2024, pp. 572–576.

[44] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, Z. Zheng, Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation, Proceedings of the ACM on Software Engineering 2 (ISSTA) (2025) 481–503.

[45] G. Sriramanan, S. Bharti, V. S. Sadasivan, S. Saha, P. Kattakinda, S. Feizi, Llm-check: Investigating detection of hallucinations in large language models, Advances in Neural Information Processing Systems 37 (2024) 34188–34216.

[46] Z. Zhang, X. Hu, H. Zhang, J. Zhang, X. Wan, Icr probe: Tracking hidden state dynamics for reliable hallucination detection in llms, arXiv preprint arXiv:2507.16488.

[47] A. B. Al-Badareen, M. H. Selamat, M. A. Jabar, J. Din, S. Turaev, Software quality models: A comparative study, in: International Conference on Software Engineering and Computer Systems, Springer, 2011, pp. 46–55.

[48] F. Yin, J. Srinivasa, K.-W. Chang, Characterizing truthfulness in large language model generations with local intrinsic dimension, in: Proceedings of the 41st International Conference on Machine Learning, 2024, pp. 57069–57084.

[49] B. Snyder, M. Moisescu, M. B. Zafar, On early detection of hallucinations in factual question answering, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 2721–2732.

[50] M. L. Siddiq, J. C. Santos, Securityeval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques, in: Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security, 2022, pp. 29–33.

[51] J. Peng, L. Cui, K. Huang, J. Yang, B. Ray, Cweval: Outcome-driven evaluation on functionality and security of llm code generation, in: 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code), IEEE, 2025, pp. 33–40.

[52] Y. Fu, E. Baker, Y. Ding, Y. Chen, Constrained decoding for secure code generation, arXiv preprint arXiv:2405.00218.

[53] M. L. Siddiq, J. C. da Silva Santos, S. Devareddy, A. Muller, Sallm: Security assessment of generated code, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops, 2024, pp. 54–65.

[54] Bandit.
URL https://bandit.readthedocs.io/en/latest/

[55] Codeql.
URL https://codeql.github.com

[56] T. T. Vu, T.-D. Bui, T.-T. Nguyen, S. Nguyen, D. H. Vo, Model-agnostic quality assessment for llm-generated code via dynamic internal representation selection.
URL https://github.com/iSE-UET-VNU/AUTOPROBE

[57] C. Team, H. Zhao, J. Hui, J. Howland, N. Nguyen, S. Zuo, A. Hu, C. A. Choquette-Choo, J. Shen, J. Kelley, et al., Codegemma: Open code models based on gemma, arXiv preprint arXiv:2406.11409.

[58] OpenAI, gpt-oss.
URL https://github.com/openai/gpt-oss

[59] Y.-S. Chuang, L. Qiu, C.-Y. Hsieh, R. Krishna, Y. Kim, J. Glass, Lookback lens: Detecting and mitigating contextual hallucinations in large language models using only attention maps, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 1419–1436.

[60] A. Kanade, P. Maniatis, G. Balakrishnan, K. Shi, Learning and evaluating contextual embedding of source code, in: Proceedings of the 37th International Conference on Machine Learning, ICML'20, JMLR.org, 2020.

[61] J. Jiang, J. Zhou, Z. Zhu, Tracing representation progression: Analyzing and enhancing layer-wise similarity, arXiv preprint arXiv:2406.14479.

[62] F. Wei, R. Keeling, N. Huber-Fliflet, J. Zhang, A. Dabrowski, J. Yang, Q. Mao, H. Qin, Empirical study of llm fine-tuning for text classification in legal document review, in: 2023 IEEE international conference on big data (BigData), IEEE, 2023, pp. 2786–2792.

[63] Y. Zhang, R. Yang, X. Xu, R. Li, J. Xiao, J. Shen, J. Han, Teleclass: Taxonomy enrichment and llm-enhanced hierarchical text classification with minimal supervision, in: Proceedings of the ACM on Web Conference 2025, 2025, pp. 2032–2042.

[64] J. Zhang, J. Huang, S. Jin, S. Lu, Vision-language models for vision tasks: A survey, IEEE transactions on pattern analysis and machine intelligence 46 (8) (2024) 5625–5644.

[65] J. Chen, Q. Yu, X. Shen, A. Yuille, L.-C. Chen, Vitamin: Designing scalable vision models in the vision-language era, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 12954–12966.

[66] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, Q. Wang, Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification, Proceedings of the ACM on Software Engineering 1 (FSE) (2024) 2332–2354.

[67] Z. Wang, K. Liu, G. Li, Z. Jin, Hits: High-coverage llm-based unit test generation via method slicing, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1258–1268.

[68] W. Sun, Y. Miao, Y. Li, H. Zhang, C. Fang, Y. Liu, G. Deng, Y. Liu, Z. Chen, Source code summarization in the era of large language models, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE Computer Society, 2024, pp. 419–431.

[69] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, A. Svyatkovskiy, Inferfix: End-to-end program repair with llms, in: Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering, 2023, pp. 1646–1656.

[70] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, L. Zhang, Z. Li, Y. Ma, Exploring and evaluating hallucinations in llm-powered code generation, arXiv preprint arXiv:2404.00971.

[71] Z. Wang, Z. Zhou, D. Song, Y. Huang, S. Chen, L. Ma, T. Zhang, Towards understanding the characteristics of code generation errors made by large language models, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE Computer Society, 2025, pp. 717–717.

[72] P. Manakul, A. Liusie, M. Gales, Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 9004–9017.

[73] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, et al., A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, ACM Transactions on Information Systems 43 (2) (2025) 1–55.

[74] S. Farquhar, J. Kossen, L. Kuhn, Y. Gal, Detecting hallucinations in large language models using semantic entropy, Nature 630 (8017) (2024) 625–630.

[75] X. Zhang, B. Peng, Y. Tian, J. Zhou, L. Jin, L. Song, H. Mi, H. Meng, Self-alignment for factuality: Mitigating hallucinations in llms via self-evaluation, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 1946–1965.

[76] J. Zhang, C. Xu, Y. Gai, F. Lecue, D. Song, B. Li, Knowhalu: Hallucination detection via multi-form knowledge based factual checking, arXiv preprint arXiv:2404.02935.

[77] C. Zhou, G. Neubig, J. Gu, M. Diab, F. Guzmán, L. Zettlemoyer, M. Ghazvininejad, Detecting hallucinated content in conditional neural sequence generation, in: Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, 2021, pp. 1393–1404.

[78] Y. Huang, J. Song, Z. Wang, S. Zhao, H. Chen, F. Juefei-Xu, L. Ma, Look before you leap: An exploratory study of uncertainty measurement for large language models, arXiv preprint arXiv:2307.10236.

[79] I. Augenstein, T. Baldwin, M. Cha, T. Chakraborty, G. L. Ciampaglia, D. Corney, R. DiResta, E. Ferrara, S. Hale, A. Halevy, et al., Factuality challenges in the era of large language models and opportunities for fact-checking, Nature Machine Intelligence 6 (8) (2024) 852–863.

[80] X. Hu, D. Ru, L. Qiu, Q. Guo, T. Zhang, Y. Xu, Y. Luo, P. Liu, Y. Zhang, Z. Zhang, Knowledge-centric hallucination detection, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 6953–6975.

[81] J. He, Y. Gong, Z. Lin, Y. Zhao, K. Chen, et al., Llm factoscope: Uncovering llms' factual discernment through measuring inner states, in: Findings of the Association for Computational Linguistics ACL 2024, 2024, pp. 10218–10230.

[82] R. Croft, D. Newlands, Z. Chen, M. A. Babar, An empirical study of rule-based and learning-based approaches for static application security testing, in: Proceedings of the 15th ACM/IEEE international symposium on empirical software engineering and measurement (ESEM), 2021, pp. 1–12.

[83] J. Zhang, H. Zhang, C. Xia, L. Sun, Graph-bert: Only attention is needed for learning graph representations, arXiv preprint arXiv:2001.05140.

[84] Y. Li, S. Wang, T. N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 292–303.

[85] T.-T. Nguyen, H. D. Vo, Context-based statement-level vulnerability localization, Information and Software Technology 169 (2024) 107406.

[86] Y. Ding, S. Suneja, Y. Zheng, J. Laredo, A. Morari, G. Kaiser, B. Ray, Velvet: a novel ensemble learning approach to automatically locate vulnerable statements, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering, IEEE, 2022, pp. 959–970.

[87] D. Hin, A. Kan, H. Chen, M. A. Babar, Linevd: Statement-level vulnerability detection using graph neural networks, in: IEEE/ACM 19th International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022, IEEE, 2022, pp. 596–607.

[88] M. Fu, C. Tantithamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 608–620.

[89] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, B. Dolan-Gavitt, Lost at c: A user study on the security implications of large language model code assistants, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 2205–2222.

[90] D. Song, Z. Zhou, Z. Wang, Y. Huang, S. Chen, B. Kou, L. Ma, T. Zhang, An empirical study of code generation errors made by large

language models, in: 7th Annual Symposium on Machine Programming, 2023.

[91] A. Mohsin, H. Janicke, A. Wood, I. H. Sarker, L. Maglaras, N. Janjua, Can we trust large language models generated code? a framework for in-context learning, security patterns, and code evaluations across diverse llms, arXiv preprint arXiv:2406.12513.

[92] C. Spiess, D. Gros, K. S. Pai, M. Pradel, M. R. I. Rabin, A. Alipour, S. Jha, P. Devanbu, T. Ahmed, Calibration and correctness of language models for code, arXiv preprint arXiv:2402.02047.

[93] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, J. Chen, Security weaknesses of copilot generated code in github, arXiv preprint arXiv:2310.02059.

[94] A. Kavian, M. M. Pourhashem Kallehbasti, S. Kazemi, E. Firouzi, M. Ghafari, Llm security guard for code, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, 2024, pp. 600–603.

[95] A. Nunez, N. T. Islam, S. K. Jha, P. Najafirad, Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing, arXiv preprint arXiv:2409.10737.